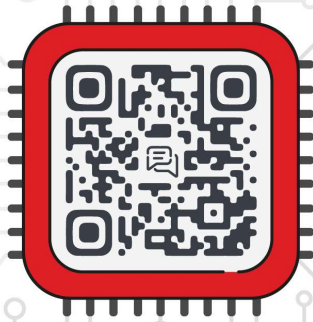




36

12

5400 sp

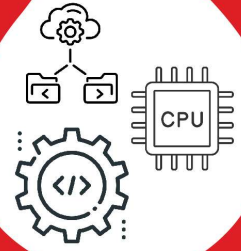
نظري

كلية الهندسة المعلوماتية

السنة الرابعة

البرمجيات الوسيطة وأنواعها في التطبيقات الموزعة

د. عمار جوخدار

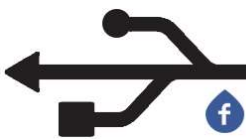
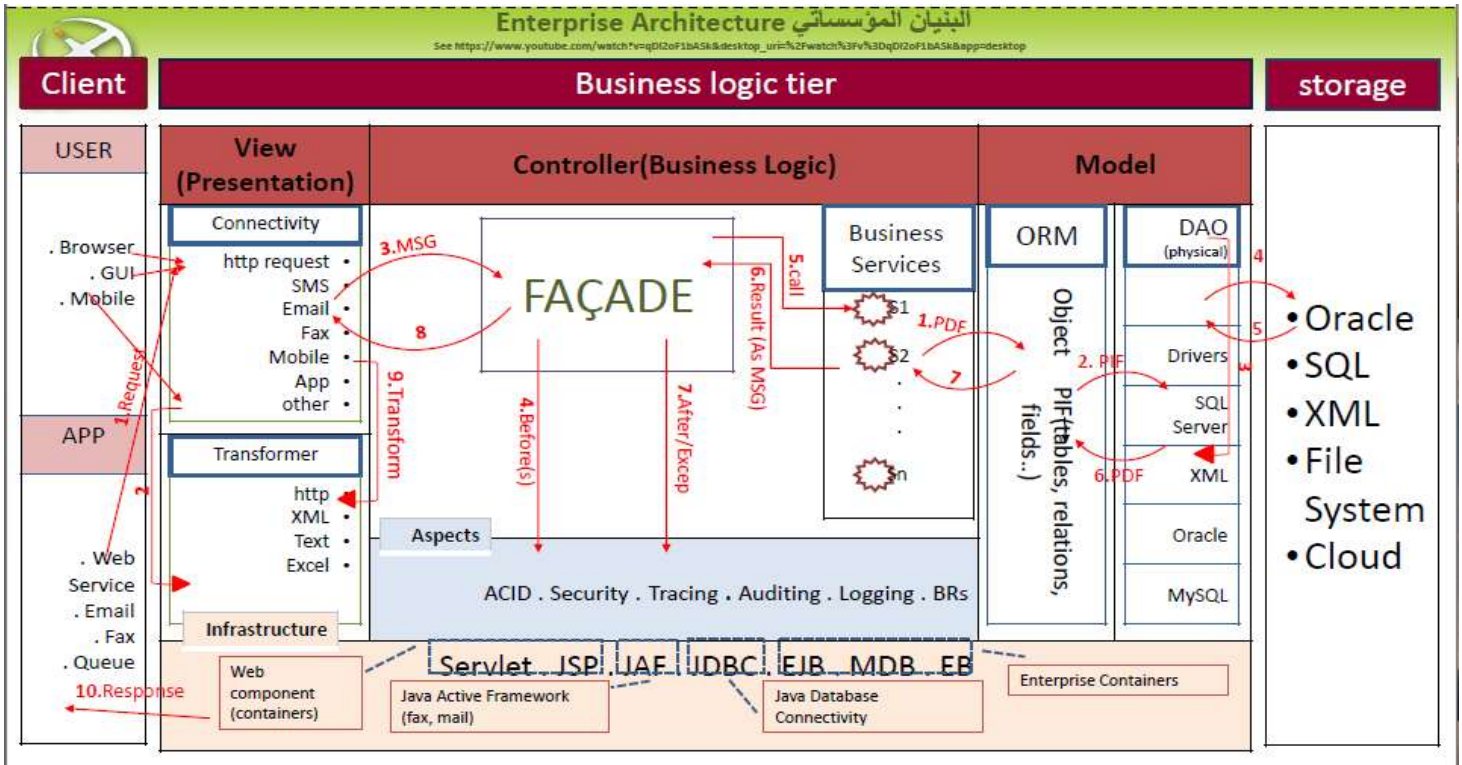


RB Informatics; 15/6/2026

برمجة تفرعية

نتناول هذه المحاضرة مفهوم البرمجيات الوسيطة ودورها في تأمين التواصل بين العمليات والخدمات المستقلة ضمن البنية البرمجي، مع توضيح الخصائص التي تستخدم لتقييمها، مثل الاستقلال عن منصة العمل، والوثوقية، والتصادفية، ودعم المناقشات وأنماط التخاطب المختلفة. كما تستعرض المحاضرة تطور أنواع البرمجيات الوسيطة بدءاً من نماذج العميل والمخدم والطابور، مروراً باستدعاء الإجراءات والأغراض عن بُعد، ووصولاً إلى خدمات الويب وإتقرايز جافا بينز. ويظهر هذا التطور كيفية انتقال وحدة التوزيع من التطبيق والرسالة والإجراء إلى الغرض والصف، ودور كل تقنية في تبسيط بناء التطبيقات الموزعة وإدارة التواصل والموارد بين مكوناتها.

تحدثنا في محاضرة سابقة عن مخطط الـ Architecture، وذكرنا أنه يمثل الصورة الشاملة للمادة؛ لذلك يجب أن نتمكن من تحديد موقع كل مفهوم ندرسه ضمن هذا البنية، ومعرفة دوره والفائدة منه. كما أشرنا إلى أن الأسهم الحمراء الموجودة في المخطط تمثل عمليات التواصل بين مكونات النظام، والتي تتم باستخدام البرمجيات الوسيطة Middleware.



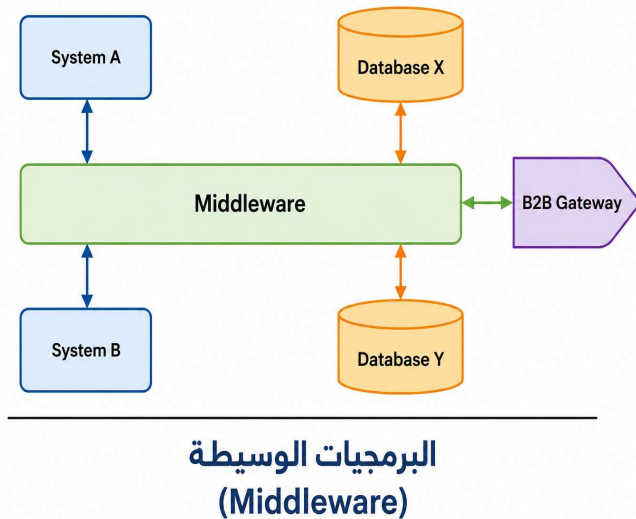
عند الحديث عن **SOA**، وهي اختصار لـ Service-Oriented Architecture أي «البنيان الموجه نحو الخدمات»، أو عن **Microservices**، فإننا نتحدث عن نظام **مقسم إلى مجموعة من الخدمات المستقلة**، إذ تطوّر كل خدمة وتعمل بصورة مستقلة، ويمكن تشغيلها أو إيقافها بصورة منفصلة، لكنها ترتبط بالخدمات الأخرى وتتواصل معها باستخدام **البرمجيات الوسيطة Middleware**. ينتج عن **ترابط** هذه الخدمات ما يُعرف بـ **Ecosystem**، ويُقصد به البيئة الشاملة للمشروع، أي النظام الكامل بجميع خدماته ومكوناته والعلاقات الموجودة بينها. وقد اشتقت كلمة Eco هنا من Ecology، أي علم البيئة، وليس من Economic؛ لذلك تعني Ecosystem حرفياً «النظام البيئي». ويمكن توضيح هذا المفهوم من خلال النظام البيئي الطبيعي؛ فالنحل والنباتات والحيوانات تبدو كأنها تعيش بصورة مستقلة، لكنها في الحقيقة **متراصة** ويعتمد بعضها على بعض، وقد يؤدي اختفاء أحد العناصر المؤثرة إلى اختلال النظام كاملاً.

وبالمثل، تبدو كل خدمة في النظام البرمجي مستقلة عن الخدمات الأخرى، لكنها تتكامل معها من خلال آليات تواصل محددة، فتعمل الخدمات مجتمعة كنظام واحد متوازن على الرغم من استقلال كل خدمة منها.

ملاحظة:

لا يمكن دائماً إدراك جميع العلاقات التي تحافظ على توازن الـ **Ecosystem**. ففي النظام البيئي الطبيعي قد تؤدي محاولة زيادة أعداد الغزلان عن طريق التخلص من الذئاب إلى نتائج عكسية؛ إذ يزداد عدد الغزلان، ثم تتراجع النباتات التي تتغذى عليها، فيختل النظام وقد يمتد التأثير إلى تغير مجاري المياه. **وبالمثل**، يعتمد استقرار النظام البرمجي على علاقات متبادلة بين خدماته، حتى لو بدت كل خدمة مستقلة عن غيرها.

فائدة:



قد توجد ضمن بنية الـ **Microservices** عدة **خدمات تقدم الوظيفة نفسها**، مثل خدمات الحصول على أسعار العملات أو الأسهم. عندها تتولى الـ **Facade** أو الـ **Broker دور الوسيط** بين الـ **Clients** والخدمات، وبين الخدمات بعضها ببعض، وكذلك بين الخدمات والـ **Aspects**، **فتبحث عن خدمة قادرة على تنفيذ الطلب وتوجهه إليها**. وبذلك لا يحتاج طالب الخدمة إلى معرفة الخدمة التي ستنفذ طلبه فعلياً، وإنما يحدد الوظيفة المطلوبة **ويتولى الوسيط اختيار مقدمها**.

يمكن للـ **Broker** اختيار الخدمة المناسبة عشوائياً، أو وفق ترتيب دوري **Round Robin**، أو باختيار الخدمة الأقل انشغالاً، أو بناءً على الكلفة. والمقصود بالاختيار بناءً على الكلفة الموازنة بين كلفة الخدمة وكلفة الوصول إليها، **فقد يكون اختيار متجر قريب بسعر أعلى** أوفر من الذهاب إلى متجر بعيد بسعر أقل إذا كانت كلفة الوقت والانتقال أكبر من قيمة التوفير. وبالطريقة نفسها ينفذ الـ **Broker** عملية **Load Balancing**، فيوزّع الطلبات بين الخدمات وفق الاستراتيجية المعتمدة، مما يمنع تركيز الحمل على خدمة واحدة.

(1) ما هي البرمجيات الوسيطة Middlewares ؟

يتكوّن التطبيق من برنامج واحد أو عدة برامج، ويتكوّن كل برنامج من عملية واحدة أو عدة عمليات Processes، بينما تتكوّن كل عملية Process من خيط تنفيذ واحد أو أكثر Threads. وفي بنية الـ Microservices تعمل كل خدمة مستقلة على شكل Process، وليست Method أو Thread، حيث تعتبر الخدمة مستقلة بحد ذاتها و ذلك أثناء التفكير بها و في بناؤها ونشرها Deployment وتشغيلها بصورة مستقلة.

تمتلك كل Process مساحة مستقلة في الذاكرة تتضمن الـ Heap، بالإضافة إلى مساحات الـ Stack الخاصة بخيوط التنفيذ التابعة لها، ولذلك لا تستطيع Process الوصول مباشرةً إلى ذاكرة Process أخرى. أما الـ Threads التي تنتمي إلى Process واحدة، فيملك كل منها Stack خاصاً به، أي الثريدات لا تشترك بال stack، لكنها تشترك جميعها في الـ Heap، وهذا ما يجعل التواصل وتبادل البيانات بينها ممكناً من خلال الذاكرة المشتركة.

تختلف آلية التواصل باختلاف موقع المكونات البرمجية:

- فإذا كانت التوابع موجودة ضمن الـ Thread نفسه، يتم التواصل بينها باستخدام Function Call، وهي عملية انتقال Jump في الذاكرة إلى تعليمات التابع المطلوب، ثم العودة Return بعد انتهاء تنفيذه، وتعد هذه الآلية الأسرع.
- وإذا كانت المكونات تعمل ضمن Threads مختلفة تابعة لـ Process نفسها، فإنها تتواصل عن طريق البيانات المشتركة في الـ Heap.
- أما الـ Processes المستقلة فلا تشترك في الذاكرة، ولذلك لا تستطيع التواصل وتبادل البيانات مباشرةً، بل تحتاج إلى برمجيات وسيطة Middlewares تتولى نقل الطلبات والبيانات بينها.

ومن أمثلة ذلك تواصل تطبيق المستخدم الموجود ضمن Process مستقلة مع البرنامج الموجود على المخدم ضمن Process أخرى من خلال HTTP Web Service. إذا بناءً على ذلك، يمكن تعريف البرمجيات الوسيطة بأنها مجموعة من البرامج والآليات التي تتيح التواصل وتبادل البيانات بين Processes مستقلة لا تشترك في الذاكرة.

فائدة:



- تعمل كل خدمة ضمن بنية الـ Microservices على شكل Process مستقلة في تحليلها وبناؤها ونشرها Deployment وتشغيلها كما تحدثنا. لذلك يمكن أن تتوقف إحدى الخدمات مع استمرار بقية الـ Ecosystem في العمل، خاصةً عند وجود خدمات أخرى قادرة على تقديم الوظيفة نفسها. تشبه هذه الحالة إغلاق متجر واحد دون توقف المنطقة كاملة، أو موت نحلة واحدة دون اختفاء مجتمع النحل. تُسمى قدرة النظام على الاستمرار رغم تعطل أحد مكوناته Fault Tolerance، أي التسامح مع الأخطاء.

(2) خصائص البرمجيات الوسيطة :

تختلف البرمجيات الوسيطة في الخصائص التي تقدمها، ويُختار النوع المناسب منها وفق حاجات النظام. ولا يشترط أن تدعم كل برمجية وسيطة جميع الخصائص الآتية، بل قد تدعم بعضها دون بعض:

- **الاستقلال عن منصة العمل Platform**
Independence: قدرة التطبيقات على التواصل رغم اختلاف أنظمة التشغيل أو لغات البرمجة أو العتاديات المستخدمة.
- **دعم الاتصال نقطة إلى نقطة Point to Point والإذاعة Broadcasting**: في الاتصال نقطة إلى نقطة تُرسل الرسالة إلى مستقبل محدد، بينما تسمح الإذاعة بإرسالها إلى عدة مستقبلين.
- **دعم التواصل المتزامن وغير المتزامن**: في التواصل المتزامن Synchronous ينتظر المرسل انتهاء تنفيذ الطلب والحصول على النتيجة، أما في التواصل غير المتزامن Asynchronous فيرسل الطلب ويتابع عمله دون انتظار انتهاء تنفيذه. ويكون التواصل غير المتزامن مناسباً للعمليات الطويلة أو التي تعتمد على شبكات أو خدمات خارجية بطيئة.
- **تنوع بنى الاتصال**: إمكانية العمل باستخدام وسائل اتصال مختلفة، مثل Sockets أو الشبكات اللاسلكية Wireless وغيرها.
- **دعم المناقلات Transactions**: قدرة البرمجية الوسيطة على إدخال مجموعة عمليات التواصل وتبادل الرسائل المرتبطة كوحدة واحدة ضمن المناقلة، بحيث تتأثر بنجاحها أو تراجعها، فإما أن تنجح كاملةً أو يلغى تأثيرها كاملاً.
- **الوثوقية Reliability**: ضمان وصول الرسالة إلى الجهة المطلوبة مرة واحدة فقط، من دون فقدانها أو تنفيذها بصورة مكررة.
- **التصاعدية Scalability**: قدرة النظام على التلاؤم مع ازدياد حركة الطلبات Traffic وعدد المستخدمين والخدمات، دون الحاجة إلى إعادة بناء النظام.
- **دعم بروتوكول للتخاطب**: توفير قواعد وصيغ محددة لتنظيم تبادل الرسائل وفهمها بين التطبيقات، بحيث لا يضطر كل تطبيق إلى تركيب البيانات Compose وتحليلها Parse بطريقة الخاصة.
- **دعم الاستدعاء بالاسم**: تمكين التطبيق من طلب الخدمة باسمها بدل الارتباط بعنوان IP محدد. يسمح ذلك للـ Broker بالبحث عن إحدى الخدمات المتاحة التي تحمل الاسم المطلوب وتوجيه الطلب إليها.

لا تكون البرمجية الوسيطة الأفضل هي التي تدعم جميع هذه الخصائص بالضرورة، وإنما التي توفر الخصائص الملائمة لحاجات النظام وطبيعة التواصل بين مكوناته.

(3) أنواع التواصل بين البرمجيات:

وجه المقارنة	EAI	ETL
Transactions	few	many
Amount of data	enormous	small
Integration	data	applications
Purpose	operations & strategic	operations
Business care	Business intelligence	IT, E-business
	Decision making	Better workflow
	One view of customer	Data entry once
Common	transformations	
	Meta data	
	connectors	
	Service oriented architectures	

نتعامل مع البرمجيات الوسيطة لنقل البيانات بين مكونات النظام، وقد تكون كمية البيانات المنقولة كبيرة أو صغيرة. كما قد تكون البيانات آنية، أي تمثل المعلومات الحقيقية في لحظة طلبها، أو قد تكون معلومات متأخرة تجمع وتُنقل لاحقاً. وبناءً على كمية البيانات وعدد عمليات النقل وزمن الحاجة إليها، والهدف من عملية التكامل، يمكن التمييز بين أسلوب ETL و EAI.



«العلم يرفعُ بيوتاً لا عمادَ لها،
والجهلُ يهدمُ بيتَ العزِّ والشَّرَفِ.»

حافظ إبراهيم

EAI	ETL
- يرمز EAI إلى Enterprise Application Integration، أي تكامل التطبيقات المؤسسية. يُستخدم هذا الأسلوب لإجراء عدد كبير من عمليات النقل، بحيث تنقل كل <u>عملية كمية صغيرة</u> من البيانات عند الحاجة If Needed. لذلك يكون مناسباً لتبادل البيانات الآنية بين التطبيقات <u>وتنفيذ العمليات التشغيلية اليومية</u>. <u>يشبه هذا الأسلوب</u> المستهلك الذي لا يشتري كمية كبيرة من البطيخ مسبقاً، بل يشتري بطيخة واحدة عند حاجته إليها و بسعرها الحالي. وبذلك تكون عمليات الشراء كثيرة، <u>لكن الكمية المنقولة في كل مرة صغيرة</u> وحديثة. - إذاً يركز EAI على <u>ربط التطبيقات بعضها ببعض</u> وتحسين سير العمل Workflow، كما يسمح بإدخال البيانات مرة واحدة ثم مشاركتها بين التطبيقات التي تحتاج إليها.	- يرمز ETL إلى Extraction, Transformation, and Load، أي استخراج البيانات وتحويلها ثم تحميلها. يُستخدم هذا الأسلوب لنقل كميات ضخمة من البيانات من خلال عدد قليل من عمليات النقل، إذ <u>تُجمع البيانات وتنقل على دفعات كبيرة</u> وفق أوقات مخطط لها مسبقاً، ولذلك قد لا تكون البيانات المنقولة آنية. <u>يشبه هذا الأسلوب</u> التاجر الذي يشتري كمية كبيرة من البطيخ تكفيه عدة أيام؛ فهو ينفذ عدداً قليلاً من عمليات الشراء، لكن كمية البضاعة في كل عملية تكون كبيرة. وخلال هذه المدة قد يتغير السعر، فيربح التاجر أو يخسر، لأن البيانات أو الأسعار التي اعتمد عليها عند الشراء قد لا تبقى آنية. - إذاً يركز ETL على <u>تكامل البيانات Data Integration</u>، ويُستخدم في العمليات التشغيلية والاستراتيجية، ولا سيما في ذكاء الأعمال Business Intelligence، <u>ودعم اتخاذ القرار، وتجميع بيانات</u> العمل من مصادر مختلفة لتكوين <u>رؤية موحدة عنه</u>.

فائدة:

- لا يُعد الاختيار بين ETL و EAI قراراً يتخذه المبرمج أثناء كتابة الكود البرمجي، بل هو قرار يتخذ في مرحلتي التحليل والتصميم وفق طبيعة النظام وحجم البيانات وعدد عمليات النقل والحاجة إلى البيانات الآنية.
- تقع على المحلل والمصمم مسؤولية اختيار أسلوب التكامل الذي يضمن نجاح النظام، بينما ينفذ المبرمج الجزء المحدد له وفق التصميم الموضوع.
- تتمثل مهمة المحلل والمصمم في تقسيم النظام الكبير إلى خدمات ومشروعات صغيرة يستطيع المبرمجون تنفيذها بصورة مستقلة، مع تحديد آليات تواصل تضمن تكامل هذه الأجزاء بعد تشغيلها.
- لا يُفترض أن يعرف كل مبرمج جميع تفاصيل النظام، لأن ربط كل جزء مباشرةً بجميع الأجزاء الأخرى يؤدي إلى زيادة العلاقات والتبعيات وتكوين نظام معقد يُعرف باسم Spaghetti System.



(4) لأنواع المختلفة للبرمجيات الوسيطة:

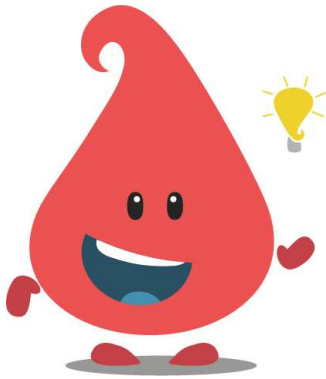
تتضمن الأنواع المختلفة للبرمجيات الوسيطة التي سندرسها في هذه المحاضرة ما يأتي:

1. Client/Server
2. Three-Tier Client/Server
3. Queue
4. Remote Procedure Call (RPC)
5. Object-Oriented Middleware، وتشمل:

- Remote Method Invocation (RMI)
- Common Object Request Broker Architecture (CORBA)
- Enterprise JavaBeans (EJB)
- Component Object Model Plus (COM+)
- Web Services

فائدة اريسيزية:

لكي تتمكن الخدمات المستقلة من التواصل، يجب تحديد بروتوكول تخاطب وواجهات برمجية APIs توضح طريقة طلب كل خدمة والبيانات التي تستقبلها والنتيجة التي تعيدها. فعلى سبيل المثال، يحدد النظام الطلب الذي يجب إرساله للحصول على أسعار العملات أو الأسهم، من دون حاجة طالب الخدمة إلى معرفة التفاصيل الداخلية لتنفيذها. يسمح ذلك لكل فريق بتطوير خدمته بصورة مستقلة مع الحفاظ على إمكانية تكاملها مع بقية النظام.



1. زبون/مخدم Client/Server:

يُعد Client/Server من أبسط أنواع البرمجيات الوسيطة، ويعتمد على استخدام Socket لربط تطبيقين مستقلين؛ أحدهما يعمل بوصفه Client والآخر بوصفه Server، ثم تُمرر البيانات المطلوبة بينهما عبر هذا الاتصال. يُوزع العمل بين الطرفين كما يأتي:

- Client: يهتم بواجهة التخاطب GUI التي يتعامل المستخدم من خلالها مع النظام، وقد يتولى أيضًا تنفيذ منطق العمل Business Logic.
- Server: يهتم بتخزين البيانات والتعامل مع قاعدة البيانات، وقد يتولى أيضًا تنفيذ منطق العمل Business Logic، وهو النموذج الأفضل من الناحية التصميمية.

المنهجية 2 Thin Client	المنهجية 1 Thick Client	
DB BL	DB	Server
GUI	GUI + BL	Client
توجد واجهة التخابر GUI فقط عند ال-Client، بينما يوجد منطق العمل Business Logic مع قاعدة البيانات عند ال-Server. يُعد هذا النموذج أفضل تصميمياً وأكثر أماناً، لأن منطق العمل يبقى على المخدم، لكنه قد يؤدي إلى زيادة الحمل وبطء الأداء؛ فال-Client يخدم مستخدماً واحداً ويملك موارد معالجة خاصة به، بينما يستقبل ال-Server طلبات عدد كبير من المستخدمين في الوقت نفسه.	توجد واجهة التخابر GUI ومنطق العمل Business Logic عند ال-Client، بينما يتولى ال-Server تخزين البيانات فقط. يخفف هذا الأسلوب الحمل عن المخدم ويستفيد من قدرة المعالجة الموجودة على جهاز المستخدم، لكنه يجعل منطق العمل موجوداً لدى العميل، مما يزيد المخاطر الأمنية ويسهل الوصول إليه أو التلاعب به.	مكان وجود ال-GUI
هنا في server يؤدي الى بطء شديد أي : مشكلة في الأداء performance	هنا في client يسهل الاختراق أي : مشكلة في الامان security	استعمال BL

لذلك تمثل المنهجتان مفاضلة بين الأمن والأداء، فوضع منطق العمل عند ال-Client يحسن توزيع الحمل لكنه يسبب مشكلة أمنية، بينما وضعه عند ال-Server يحسن الحماية لكنه يزيد الحمل على المخدم.

مثال على Client/Server

يوضح المثال طريقة إنشاء اتصال بين تطبيقين باستخدام Socket، بحيث يبدأ ال-Server بالعمل و ينتظر طلبات الاتصال، بينما ينشئ ال-Client اتصالاً معه باستخدام عنوانه ورقم المنفذ المتفق عليه.

■ أولاً: عمل ال-Client

```
Class client{
    Public static void main(){
        ClientSocket cs = new ClientSocket(ip,4000)
        Is.os;
        dealWith(os,s)
    }
}
```

ينشئ ال-Client كائناً من ClientSocket، ويحدد عنوان IP الخاص بالمخدم ورقم المنفذ 4000:

```
ClientSocket cs = new ClientSocket(ip,
4000);
```

بعد إنشاء الاتصال، يحصل الـ Client على مجري الإدخال والإخراج OutputStream و InputStream. ثم يستخدمهما لإرسال البيانات إلى الـ Server واستقبال النتائج منه. يتولى التابع dealWith معالجة عملية التواصل باستخدام هذين المجريين.

■ ثانياً: عمل الـ Server:

```
Class Server{
    Public static void main(){
        ServerSocket ss = new ServerSocket(4000);
        ServerSocket ss = new ServerSocket(4000);
        while (true){
            Socket cs = ss.accept();
            InputStream is = cs.getInputStream();
            OutputStream os = cs.getOutputStream();
            dealWith(is,os);
        }
    }
}
```

ينشئ الـ Server كائناً من ServerSocket ويربطه المنفذ 4000:

```
ServerSocket ss = new ServerSocket(4000);
```

■ بعد ذلك يدخل في حلقة مستمرة ليستقبل طلبات الاتصال المتتالية. عند وصول طلب جديد، يستدعي التابع accept للحصول على Socket خاصة بالـ Client المتصل:

Socket cs = ss.accept();

■ ثم يحصل الـ Server على InputStream لقراءة البيانات القادمة من الـ Client، و OutputStream لإرسال النتيجة إليه، ويمررهما إلى التابع dealWith لتنفيذ المعالجة المطلوبة.

■ يمثل هذا المثال آلية التواصل الأساسية في نموذج Client/Server: يفتح الـ Client اتصالاً مع المخدم، ويرسل الطلب من خلال الـ Socket، فيقرأه الـ Server ويعالجه ثم يعيد النتيجة عبر الاتصال نفسه.

↪ محدودية Client/Server

نعم	الاستقلال عن منصة العمل
point to point only	دعم Broadcasting/ point to point
لا	دعم Transactions
يعتمد على المبرمج	الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط
لا	التصعديّة أو التلاؤم مع تزايد الحركة Traffic
متزامن فقط	دعم التخاطب المتزامن وغير المتزامن.
لا	دعم بروتوكول تخاطب
لا	دعم الاستدعاء بالاسم
socket	تنوع بنى الاتصال : امكّانة العمل عن طرّق wireless, sockets, ... etc.

يمكن تقييم نموذج Client/Server وفق خصائص البرمجيات الوسيطة كما يأتي:

- **الاستقلال عن منصة العمل:** يدعم هذا النموذج الاستقلال عن نظام التشغيل و منصة العمل، إذ يمكن استخدام Socket للتواصل بين تطبيقات تعمل على منصات مختلفة.
- **نمط الاتصال:** يدعم الاتصال نقطة إلى نقطة Point to Point فقط، لأن Client يفتح اتصالاً مباشراً مع Server محدد، ولا يدعم الإذاعة Broadcasting التي تسمح بإرسال الرسالة إلى عدة مستقبلين.
- **المناقلات Transactions:** لا يدعم المناقلات، ولذلك لا تكون عملية إرسال البيانات عبر Socket جزءاً تلقائياً من المناقلة.
- **الوثوقية Reliability:** تعتمد على المبرمج؛ إذ يجب عليه كتابة الآليات التي تضمن عدم فقدان الرسالة أو تكرار تنفيذها.
- **التصاعدية Scalability:** لا يتلاءم هذا النموذج مع الزيادة الكبيرة في حركة الطلبات Traffic. فالـ Client يتصل بعنوان IP محدد، وإذا ازداد عدد المستخدمين أو المخدمات، فقد يلزم تعديل البرنامج ليتعامل مع عناوين واتصالات إضافية.
- **نمط التخاطب:** يدعم التخاطب المتزامن Synchronous فقط؛ إذ يرسل Client الطلب إلى Server وينتظر وصول الجواب قبل متابعة التنفيذ.
- **بروتوكول التخاطب:** لا يوفر بروتوكولاً يحدد بنية البيانات المتبادلة ومعناها. يستطيع المبرمج إرسال البيانات بالصيغة التي يريدها، لكنه يصبح مسؤولاً عن تركيبها Compose قبل الإرسال وتحليلها Parse عند الاستقبال.
- **يشبه الـ Socket في هذه الحالة خدمة البريد:** فالبريد يفهم العنوان المكتوب على الرسالة ويوصلها إلى المستقبل، لكنه لا يفهم محتوى الرسالة ولا يضمن أن المستقبل يستطيع قراءته. وبالمثل، تنقل الـ Socket البيانات، لكن على التطبيقين الاتفاق مسبقاً على صيغة مشتركة لفهمها. قد تصبح عملية التحليل Parsing معقدة، خاصةً عندما تكون بنية البيانات كبيرة أو متعددة الأشكال.
- **الاستدعاء بالاسم:** لا يدعم الاستدعاء بالاسم، بل يتم الاتصال باستخدام عنوان IP ورقم المنفذ. لذلك يجب على Client معرفة موقع الـ Server الذي يريد الاتصال به.
- **تنوع بنى الاتصال:** يقتصر هذا النموذج على الاتصال باستخدام Sockets، ولا يوفر تنوعاً في بنى الاتصال.
- **بناءً على ذلك،** يوفر Client/Server اتصالاً مباشراً وبسيطاً بين تطبيقين، لكنه لا يدعم عدداً من الخصائص المهمة، مثل Broadcasting والمناقلات والتصاعدية والتخاطب غير المتزامن وبروتوكول التخاطب والاستدعاء بالاسم.

فائدة:

- في الاستدعاء بالعنوان يجب على طالب الخدمة معرفة عنوان IP الخاص بمقدمها والاتصال به مباشرةً. أما في الاستدعاء بالاسم، فيطلب التطبيق الخدمة باسمها، مثل طلب خدمة لبيع منتج معين دون تحديد المتجر الذي سيقدمها. عندها يستطيع Broker البحث عن إحدى الخدمات المتاحة التي تحمل الاسم المطلوب وتوجيه الطلب إليها.

■ يسمح الاستدعاء بالاسم بتغيير مقدم الخدمة أو إضافة عدة خدمات تقدم الوظيفة نفسها دون اضطراب Client إلى معرفة عناوينها. أما في نموذج Client/Server المعتمد على Socket، فيرتبط Client بعنوان IP محدد، ولذلك لا تتوافر هذه المرونة.

فائدة: الصورة المتقدمة للـ Ecosystem:

■ تقوم الفكرة المتقدمة للـ Ecosystem على نشر خدمة جديدة بحيث تستطيع بقية التطبيقات اكتشافها والتعامل معها تلقائياً عن طريق Broker، بدل ربط كل تطبيق بها يدوياً. يشبه ذلك افتتاح متجر جديد ضمن سوق؛ فإذا كانت هناك حاجة إلى خدمته، تبدأ الجهات الأخرى بالتعامل معه ويستمر، وإذا لم توجد حاجة إليه يخرج من النظام تدريجياً. وبهذه الطريقة يفترض أن يتوازن الـ Ecosystem وفق العرض والطلب، مع الإشارة إلى أن تحقيق هذا المستوى من الاكتشاف والتكامل التلقائي ما يزال أصعب من صورته النظرية.

2. Three-Tier Client/Server:

انتقل نموذج Client/Server إلى بنية Three-Tier Client/Server من خلال إضافة طبقة مستقلة لمنطق العمل بين Client وقاعدة البيانات. وبذلك يتكون النظام من ثلاث طبقات:

■ **طبقة Client:** تحتوي على واجهة التخابر GUI التي يتعامل المستخدم من خلالها مع النظام، وترسل طلباته إلى الطبقة الوسطى.

■ **طبقة Business Logic Server:** تمثل الطبقة الوسطى، وتحتوي على طبقة العرض Presentation، ومنطق العمل Business Logic المسؤول عن معالجة الطلبات وتطبيق قواعد النظام، بالإضافة إلى ORM و DAO المسؤولين عن الوصول إلى البيانات والتعامل معها.

■ **طبقة Storage:** تحتوي على قاعدة البيانات DB التي تخزن البيانات المشتركة.

■ **يطلق على هذا التقسيم اسم Three-Tier Architecture أو Three-Tier Middleware، وهو البنية** Enterprise Architecture المكوّن من طبقة العميل، وطبقة مخدّم منطق العمل، وطبقة التخزين.

■ **في هذا النموذج لم يعد منطق العمل موجوداً عند Client أو مدمجاً مع قاعدة البيانات، بل أصبح** في طبقة مستقلة تمثل طبقة المعالجة أو «الذكاء»؛ فهي تعرف كيفية تنفيذ العمليات، بينما تحصل على البيانات اللازمة من طبقة التخزين.

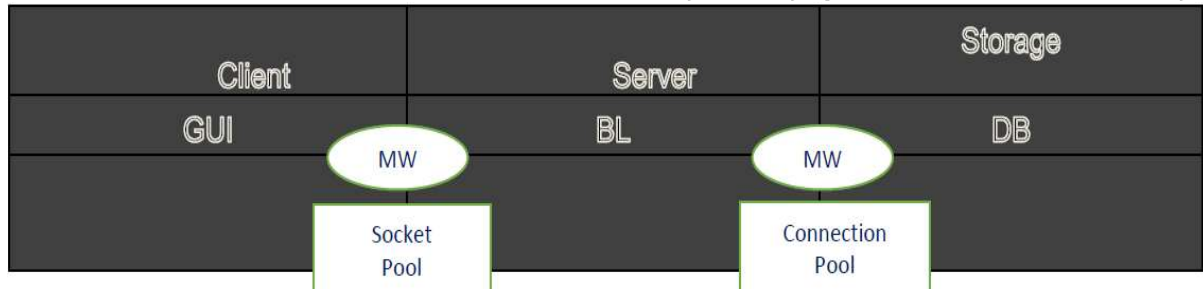
■ **يمكن تشغيل عدة نسخ من طبقة Business Logic بالتوازي،** لأنها تنفذ القواعد نفسها على طلبات مختلفة، في حين يكون تكرار قاعدة البيانات أكثر صعوبة بسبب ضرورة المحافظة على نسخة موحدة ومتناسقة من البيانات. لذلك يمكن إضافة عدة مخدّمات لمنطق العمل لمعالجة عدد أكبر من الطلبات، مع استمرارها جميعاً في التعامل مع البيانات المشتركة نفسها.

■ **يشبه ذلك وجود عدة Cashiers في متجر واحد،** إذ يستطيع كل موظف خدمة زبون مختلف وتنفيذ عملية البيع بالتوازي مع الآخرين، بينما يستخدم الجميع الأسعار والمخزون وبيانات الزبائن نفسها. وبذلك يتحقق الـ Parallel Processing في طبقة منطق العمل، وليس من خلال إنشاء نسخة مستقلة من البيانات لكل موظف.

فائدة:

يمكن توضيح مبدأ فصل منطق العمل عن البيانات من خلال مثال المتجر القديم؛ فقد كان صاحب المتجر يحفظ الأسعار والموردين وديون الزبائن في ذاكرته، ولذلك كان العمل يتوقف عند غيابه. عندما نُقلت هذه المعلومات إلى دفتر مشترك، أصبح بإمكان شخص آخر متابعة العمل، ثم أصبح بإمكان عدة أشخاص العمل معاً اعتماداً على البيانات نفسها. يمثل الموظفون هنا نسخاً متعددة من طبقة Business Logic، بينما يمثل الدفتر قاعدة البيانات المشتركة.

يتم التواصل بين الطبقات باستخدام برمجيات وسيطة:



- بين Client وطبقة Business Logic يوجد [Socket Pool](#)، وهو مجموعة من موارد الاتصال الجاهزة لاستقبال طلبات المستخدمين وتوزيعها على خيوط التنفيذ المتاحة.
- بين طبقة Business Logic وقاعدة البيانات يوجد [Connection Pool](#)، وهو مجموعة من اتصالات قاعدة البيانات التي تُنشأ مسبقاً ويُعاد استخدامها بدل إنشاء اتصال جديد مع كل طلب.
- يسمح استخدام [Pools بإعادة استعمال الموارد](#)، وتنفيذ عدة طلبات بالتوازي، والتحكم في عدد الاتصالات وخيوط التنفيذ بما يتناسب مع الذاكرة وقدرة المعالجة المتاحة.

تذكرة:

- تُحدّد أعداد Threads والاتصالات داخل Pools وفق موارد النظام، مثل الذاكرة وقدرة المعالج وعدد الطلبات المتوقع. عند وصول طلب متزامن، يخصص Web Server خيط تنفيذ متاحاً لمعالجته، ثم يُعاد الخيط إلى Pool بعد انتهاء الطلب ليستخدم من جديد. وإذا كانت جميع الخيوط مشغولة ولم يتوافر خيط خلال مدة الانتظار المحددة، يحدث Timeout. أما في التخاطب غير المتزامن، فيُرسَل الطلب إلى Queue وينتهي خيط التنفيذ من دون انتظار النتيجة، ثم تتابع النتيجة لاحقاً باستخدام Promise.
- لذلك ممنوع منا باتاً أن ينشئ المبرمج Threads يدوياً داخل منطق التطبيق، لأن ذلك يتجاوز إدارة الموارد التي حددها تصميم النظام، وقد يؤدي إلى استهلاك غير مضبوط للذاكرة والمعالج. تتولى الـ Containers وإدارة Pools خيوط التنفيذ والاتصالات، بينما يكتب المبرمج منطق الخدمة المطلوب.

Three-Tier Client/Server محدودية

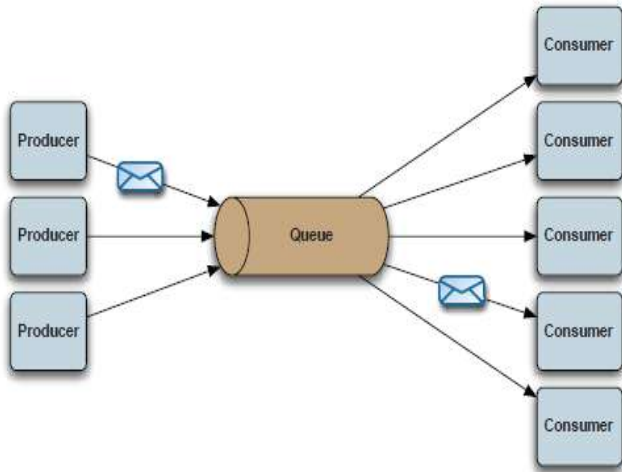
نعم	الاستقلال عن منصة العمل
point to point only	دعم Broadcasting/ point to point
لا	دعم Transactions
يعتمد على المبرمج	الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط
نعم	التصاعدية أو التلاؤم مع تزايد الحركة Traffic
متزامن فقط	دعم التخاطب المتزامن وغير المتزامن.
لا	دعم بروتوكول تخاطب
لا	دعم الاستدعاء بالاسم
socket	تنوع بنى الاتصال : امكّانة العمل عن طرّق etc, ... wireless, sockets

أصبح الأمن من مسؤولية طبقة BL وبالتالي أصبح هناك صعوبة أكبر بكثير في البرمجة مقارنة مع client/server.

يمكن تحقيق بنية Three-Tier Client/Server باستخدام تقنيات مختلفة، مثل HTTP و Web Services. وتشبه خصائص هذا النموذج خصائص Client/Server، لكن الفرق الأساسي بينهما أن Three-Tier Client/Server يدعم التصاعدية نتيجة فصل طبقة Business Logic وإمكانية تكرارها لتخديم عدد أكبر من الطلبات. وفقاً لخصائص البرمجيات الوسيطة، يحقق هذا النموذج ما يأتي:

- يدعم الاستقلال عن منصة العمل.
- يدعم الاتصال نقطة إلى نقطة Point to Point فقط، ولا يدعم Broadcasting.
- لا يدعم المناقلات Transactions.
- تعتمد الوثوقية على المبرمج.
- يدعم التصاعدية والتلاؤم مع تزايد الحركة Traffic.
- يدعم التخاطب المتزامن Synchronous فقط.
- لا يدعم بروتوكول تخاطب.
- لا يدعم الاستدعاء بالاسم.
- يعتمد على Sockets في بنية الاتصال.
- كما أصبح الأمن من مسؤولية طبقة Business Logic، مما جعل البرمجة في هذا النموذج أكثر صعوبة من نموذج Client/Server.

"لا تجعل خوفك من الفشل يمنعك من المحاولة."



الـ Queue هو برنامج وسيط بسيط لا يستعمل

بروتوكولات نظام التشغيل ولا يستدعي تعليمات النظام، ويوفر بصورة أساسية التخاطب غير المتزامن Asynchronous Communication.

في التخاطب المتزامن Synchronous يرسل طالب

الخدمة طلبه و ينتظر انتهاء تنفيذه والحصول على النتيجة. يناسب هذا الأسلوب الخدمات القصيرة، مثل شراء منتج جاهز من متجر، حيث يُنفذ الطلب وتظهر النتيجة مباشرةً.

أما في التخاطب غير المتزامن Asynchronous،

فيرسل طالب الخدمة طلبه ويتابع عمله من دون انتظار انتهاء التنفيذ. يناسب هذا الأسلوب الخدمات الطويلة، مثل طلب تفصيل ثوب من الخياط، إذ لا يبقى الزبون منتظرًا حتى انتهاء العمل، بل يترك الطلب ويعود لاحقًا لاستلامه.

فائدة:

يجب اختيار نمط التخاطب وفق طبيعة الخدمة ومدتها. فإذا كان تنفيذ مرحلة أبطأ من المرحلة السابقة لها، فلا يُشترط أن تتوقف جميع المراحل بانتظارها.

يُفضّل أيضًا استخدام التخاطب غير المتزامن عند التعامل مع خدمات خارجية بطيئة، مثل إرسال SMS أو

Email عبر شبكة خارجية. فلا ينبغي أن تتوقف مناقلة أساسية، مثل عملية بيع أو شراء، بانتظار إرسال رسالة إعلامية قد تتأخر بسبب بطء الشبكة أو توقف خدمة الإرسال. تُنفذ العملية الأساسية أولاً، بينما يوضع طلب الإرسال في الـ Queue ليُنَفَّذ لاحقًا.

تُعد تقنية Queue من التقنيات الناضجة، لأنها استُخدمت واختُبرت في أنظمة ومجالات تطبيقية متعددة،

وأصبحت آلية موثوقة لتبادل الرسائل بين التطبيقات. ولا تشترط هذه التقنية أن تكون التطبيقات مبنية وفق البرمجة غرضية التوجه أو باستخدام تقنيات حديثة؛ لأن التواصل يتم من خلال إرسال الرسائل إلى الطابور واستقبالها منه، دون الحاجة إلى معرفة البنية الداخلية لكل تطبيق. لذلك يمكن استخدام الـ Queue وسيطاً يربط نظاماً قديماً Legacy System بتطبيق حديث، ما دام الطرفان قادرين على إرسال الرسائل واستقبالها بالصيغة المتفق عليها.

لكن لا يوجد معيار موحد تلزم به جميع منتجات الـ Queue، إذ قد يستخدم كل منتج واجهات وبروتوكولات

وصيغ رسائل خاصة به. لذلك لا يمكن عادةً ربط منتجين مختلفين مباشرةً أو إرسال رسالة من أحدهما إلى الآخر دون استخدام محوّل أو وسيط إضافي. ومن المنتجات الصناعية المعروفة في هذا المجال BEA MessageQ و IBM MQSeries.

- يُعد الاختيار بين التخابر المتزامن Synchronous وغير المتزامن Asynchronous قراراً تصميمياً يعتمد على مدة تنفيذ الخدمة، وسرعة الشبكة، وعلاقة العملية بالمناقلة الأساسية. تُنفذ الخدمات القصيرة التي تتطلب نتيجة فورية بصورة متزامنة، بينما تُرسل الخدمات الطويلة أو الخارجية التي لا يجب أن توقف العمل الأساسي بصورة غير متزامنة.
- تختلف سرعة تنفيذ الطلب باختلاف مكان الخدمة؛ فالعمليات المحلية Local أسرع من العمليات المنفذة عبر شبكة محلية LAN، وهذه بدورها أسرع من العمليات التي تمر عبر شبكة واسعة WAN أو خدمة خارجية. لذلك يُفضّل تنفيذ العمليات البطيئة التي لا تؤثر في نجاح المناقلة الأساسية بصورة غير متزامنة، مثل إرسال رسالة SMS أو Email، حتى لا يؤدي بطء الشبكة أو توقف خدمة الإرسال إلى تعطيل عملية البيع أو الشراء.

⚡ خصائص الطابور Queue

الاستقلال عن منصة العمل	يتوفر على منصات Unix - IBM/OS2 - Windows NT
دعم Broadcasting/ point to point	نعم
دعم Transactions	نعم
الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط	يضمن وصول الرسائل وذلك بالمحافظة على نسخة منها على القرص الصلب لا تهدم ما لم تعالج الرسالة.
التصعيدية أو التلاؤم مع تزايد الحركة Traffic	نعم
دعم التخابر المتزامن وغير المتزامن.	نعم
دعم بروتوكول تخاطب	لا
دعم الاستدعاء بالاسم	لا
تنوع بنى الاتصال : امكانّة العمل عن طرّق etc, ... wireless, sockets	نعم
يسمح بتحقيق ممتاز لنموذج Producer/Consumer بدون أي جهد.	

- يُعد الـ Queue تحقيقاً مباشراً لنموذج [Producer/Consumer](#) ويمكن استخدامه أيضاً لتطبيق مبدأ Publish/Subscribe. يرسل الـ Producer الرسائل إلى الطابور، بينما يسجل الـ Consumer اشتراكه Subscription في أنواع الرسائل التي يستطيع معالجتها.
- يمكن إعطاء كل رسالة نوعاً Message Type أو وسماً Tag، مثل رسائل البيع أو الشراء أو إرسال البريد، ثم يوزع الـ Queue الرسالة على أحد الـ Consumers المشتركين بهذا النوع. وبذلك يعمل بوصفه Broker بين الجهات التي تنتج الطلبات والخدمات التي تنفذها.

- **عند وجود عدة Consumers مشتركين ضمن المجموعة نفسها**، تُسلّم كل رسالة إلى أحدهم لمعالجتها. أما في حالة Broadcasting، فتصل نسخة من الرسالة إلى جميع الجهات المشتركة بها.
- **يمكن تقييم الـ Queue وفق خصائص البرمجيات الوسيطة كما يأتي:**
- **الاستقلال عن منصة العمل:** يتوافر على منصات مختلفة، مثل Unix و IBM/OS2 و Windows NT.
- **أنماط الاتصال:** يدعم الاتصال نقطة إلى نقطة Point to Point، كما يدعم Broadcasting؛ إذ يمكن إرسال الرسالة إلى جميع الـ Consumers المشتركين بها.
- **المناقشات Transactions:** يُعدّ الـ Queue أول نوع من البرمجيات الوسيطة التي درسناها يدعم المناقشات، بشرط أن يتم التواصل بين جميع الأطراف المشاركة باستخدامه.
- **أما عند استخدام Socket أو HTTP أو SOAP أو Web Service خارج إطار مناقلة** موزعة مشتركة، فإن تنفيذ Rollback يتراجع عن التعديلات المحلية فقط، ولا يلغي تلقائيًا العمليات المنفذة في الأنظمة الأخرى.
- **فمثلاً، عند نقل مبلغ بين بنكين**، قد يُسحب المبلغ من البنك الأول ولا يُضاف إلى الثاني بسبب انقطاع الاتصال. يؤدي ذلك إلى عدم اتساق البيانات، وقد يضطر فريق العمل إلى مراجعة قواعد البيانات وتتبع العمليات وتصحيحها يدويًا. لذلك يجب أن تدخل الأطراف جميعها ضمن آلية مناقلة مشتركة حتى تُنفذ العمليات كلها أو يُلغى تنفيذها جميعًا عند حدوث خطأ.
- **الوثوقية Reliability:** يضمن وصول الرسالة مرة واحدة فقط، إذ يحتفظ بنسخة منها على القرص الصلب ولا يحذفها حتى تُعالج بنجاح. لذلك لا تضع الرسالة عند انقطاع الكهرباء أو توقف النظام، بل تُستكمل معالجتها بعد إعادة التشغيل. وإذا فشل أحد الـ Consumers في تنفيذها، تعود إلى الـ Queue لمعالجتها Consumer آخر.
- **التصاعدية Scalability:** يدعم التلاؤم مع تزايد حركة الطلبات، إذ يمكن تشغيل الـ Producers والـ Consumers على عدة Servers، كما يمكن زيادة عدد الـ Consumers أو الـ Threads عند ازدياد الحمل وتقليلها عند انخفاضه.

فائدة:

- عندما يستقبل الـ Web Server طلبًا متزامنًا، يختار Thread متاحًا من مجموعة الـ Threads لمعالجته. فإذا كانت جميعها مشغولة، ينتظر الطلب مدة محددة، ثم يحدث Timeout إذا لم يتوافر Thread لخدمته. أما في التخاطب غير المتزامن، فيوضع الطلب في الـ Queue ويُحرر الـ Thread بدل بقاءه منتظرًا، ثم يعالج أحد الـ Consumers الرسالة لاحقًا. ويمكن زيادة عدد الـ Consumers أو الـ Threads عند ارتفاع الضغط وتقليله عند انخفاضه، من دون تغيير الجهة التي تنتج الطلبات.
- **نمط التخاطب:** يدعم التخاطب المتزامن وغير المتزامن معًا. في التخاطب المتزامن ينتظر المرسل تنفيذ الطلب وإعادة النتيجة، وقد يحدث Timeout إذا لم يتوافر Consumer لمعالجته. أما في التخاطب غير المتزامن يرسل المنتج الطلب ويتابع عمله، ثم تُعالج الرسالة لاحقًا. ويمكن معرفة النتيجة من خلال بيانات محفوظة تعبّر عن حالة التنفيذ، مثل Pending ثم Executed. ويرتبط ذلك بمفهوم Promise الذي يمثل نتيجة ستصبح متاحة لاحقًا، من دون بقاء الـ Thread المرسل منتظرًا.

فائدة:

- عند إرسال طلب غير متزامن لا يبقى الـ Thread الذي أرسل الطلب منتظراً، بل تُتابع نتيجة التنفيذ من خلال البيانات. فمثلاً، عند طلب إرسال Email توضع رسالة في الـ Queue وتكون حالتها Pending، ثم يحدث الـ Consumer حالتها إلى Executed بعد إرسال البريد بنجاح. يستطيع التطبيق لاحقاً فحص هذه الحالة لمعرفة ما إذا نُفذت العملية، حتى لو انتهى الـ Thread أو توقفت الـ Process التي أرسلت الطلب. تشبه هذه الآلية مفهوم Promise؛ إذ تمثل نتيجة ستصبح متاحة لاحقاً من دون إبقاء المرسل منتظراً.
- يجب التمييز في العمليات غير المتزامنة بين تاريخ إرسال الطلب وتاريخ تنفيذه فعلياً. فإذا كانت هناك مدة تبدأ بعد إتمام العملية، مثل منح المستخدم خمسة عشر يوماً لتنفيذ إجراء لاحق، فيجب حسابها من تاريخ التنفيذ المسجل عند انتقال الحالة إلى Executed، وليس من تاريخ وضع الرسالة في الـ Queue.

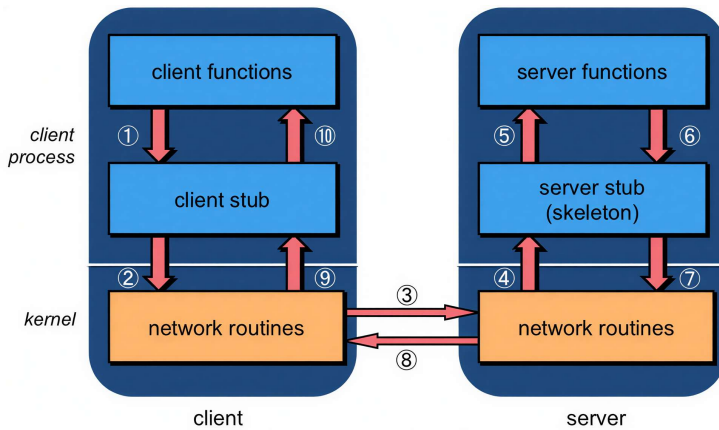
- **بروتوكول التخاطب:** لا يدعم بروتوكولاً يحدد بنية محتوى الرسالة، فالـ Queue ينقل الرسالة من دون فهمها، ويجب أن يتفق الـ Producer والـ Consumer على طريقة تفسيرها. يُعد ذلك ميزة عند ربط تطبيقات قديمة وحديثة لا تستخدم صيغة موحدة، لكنه يجعل مسؤولية فهم محتوى الرسالة على التطبيقات نفسها.
- **الاستدعاء بالاسم:** لا يدعم الاستدعاء بالاسم مباشرة، لكن يمكن إضافة طبقة فوقه، مثل Message-Driven Bean، لتوفير هذه الخاصية.
- **تنوع بنى الاتصال:** يدعم بنى اتصال متعددة، مثل Sockets والاتصالات اللاسلكية Wireless وغيرها.

فائدة:

- ظهرت JavaScript أساساً كلغة Client-Side تعمل وفق نموذج Mono-Thread، ولذلك تعتمد على Queue لتنظيم المهام وتنفيذ العمليات غير المتزامنة دون إيقاف الصفحة. كما سمحت بتعديل عناصر HTML ديناميكياً باستخدام DOM، وإرسال الطلبات وتحديث أجزاء من الصفحة باستخدام AJAX من دون إعادة تحميلها كاملة. وعند استخدام JavaScript على جانب المخدم من خلال Node.js استمر الاعتماد على الطوابير والتخاطب غير المتزامن في معالجة الطلبات.
- يسمح الـ Queue بتحقيق نموذج Producer/Consumer من دون أن يبرمج المطور بنفسه آليات التزامن وإدارة التنافس بين الـ Consumers. فهو ينظم توزيع الرسائل، ويعيد الرسالة إلى الطابور إذا فشل أحد الـ Consumers في تنفيذها، ويتيح زيادة عدد المنفذين وفق الضغط. كما أن بناء Queue موثوق من الصفر عملية معقدة تقارب في صعوبتها بناء قاعدة بيانات؛ لذلك يُفضل استخدام منتج ناضج بدل إعادة برمجة هذه الآليات. ولهذا نصفه بأنه «ملك البرمجيات الوسيطة»، فالتطبيقات الحديثة لا تكاد تخلو من Queue، لما يقدمه من حلول لمشكلات التوازي والوثوقية واستقرار النظام.

"الإنجاز الحقيقي هو أن تتغلب على نفسك قبل أن تتغلب على الصعوبات."

4. RPC - Remote Procedure Call



يسمح الـ Queue بمكاملة التطبيقات الموجودة مسبقًا، لأنه لا يفرض قيودًا على بنية الاتصال أو محتوى الرسائل. لكن عند تطوير تطبيق جديد وتوزيعه على عدة آلات، تصبح كتابة كود برمجي، إرسال الرسائل وتحليلها يدويًا عملية معقدة، ولذلك نحتاج إلى آلية أسرع وأكثر شفافية.

يتكون التطبيق في الحالة التقليدية من:

- **Client:** يمثل الإجراء الرئيسي Main Procedure الذي يطلب تنفيذ العمل.
- **Server:** يمثل الإجراء Procedure الذي يقدم الخدمة وينفذ العمل المطلوب.

عندما يكون الـ Client و الـ Server ضمن Process واحدة، يستدعي الـ Client التابع مباشرةً باسمه ويمرر له الـ Parameters. أما بعد توزيع التطبيق على عدة Processes، فيجب تجهيز اسم التابع ومعاملاته ضمن رسالة، وإرسالها عبر الشبكة، ثم يقوم الطرف الآخر بتحليل الرسالة واستخراج اسم التابع ومعاملاته، وتنفيذ التابع، وتجهيز النتيجة وإعادتها إلى الطرف المستدعي. يؤدي تنفيذ هذه الخطوات يدويًا باستخدام الـ Socket أو الـ Queue إلى زيادة حجم الكود وتعقيده.

جاء RPC لتوفير استدعاء تابع موجود ضمن Process أخرى بالطريقة نفسها تقريبًا التي يُستدعى بها التابع المحلي. يحدد المبرمج الخدمة والتابع المطلوب ومعاملاته، بينما تتولى البرمجية الوسيطة تجهيز الرسالة، وإرسالها عبر الشبكة، وإعادة بناء الطلب، وتنفيذ التابع، ثم إعادة النتيجة. وبذلك تحقق RPC شفافية الاستدعاء؛ فلا يحتاج المبرمج إلى كتابة تفاصيل الاتصال أو تركيب الرسائل وتحليلها بنفسه. ويرمز RPC إلى Remote Procedure Call، أي «استدعاء إجراء عن بُعد»، و **هو أول نوع من البرمجيات الوسيطة المدروسة يقدم بروتوكول تخاطب.**

يمكن أن يُكتب الـ Client بلغة برمجة تختلف عن لغة الـ Server، لكن لغات البرمجة لا تستخدم التسميات والأنماط نفسها؛ فقد يقابل النوع int في إحدى اللغات النوع integer أو long في لغة أخرى، كما قد يقابل char* في C++ النوع String في Java. لذلك تستخدم RPC لغة عامة تسمى IDL، وهي اختصار لـ Interface Definition Language. تصف هذه اللغة واجهة المخدم والتوابع التي يقدمها وأنماط الـ Parameters والنتائج، كما تحدد عملية الـ Mapping بين أنماط البيانات في اللغات المختلفة. وبذلك يستطيع Client مكتوب بلغة معينة استدعاء تابع موجود على Server مكتوب بلغة أخرى.

آلية عمل RPC:

عند استخدام الـ Queue أو الـ Socket، يكون الكود البرمجي اللازم للتخاطب جزءًا من كود الـ Client، ولذلك يجب على المبرمج كتابة خطوات تجهيز اسم التابع والـ Parameters ضمن رسالة، وإرسالها عبر الشبكة، ثم تحليل الجواب واستخراج النتيجة منه. يؤدي ذلك إلى زيادة حجم الكود وتعقيده، مع أن الغاية الأساسية من التواصل بين التطبيقين هي استدعاء خدمة موجودة لدى الطرف الآخر.

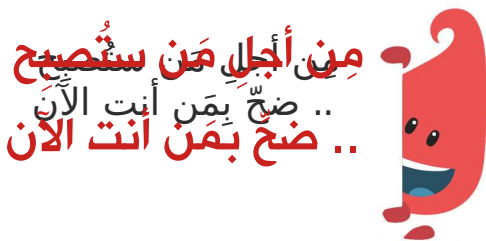
أما في RPC، فلا يكتب المبرمج تفاصيل التخابر يدوياً، ولا يظهر له فرق واضح بين استدعاء تابع محلي واستدعاء تابع موجود ضمن Process أخرى؛ إذ تصف لغة IDL واجهة المخدم والتوابع التي يقدمها وأنماط معاملاتها ونتائجها، ثم يولد الـ Compiler آلياً الكود البرمجي اللازم لتنفيذ التخابر. تتم عملية الاستدعاء وفق المراحل الآتية:

1. يستدعي الـ Client التابع المطلوب باسمه ويمرر إليه الـ Parameters كما لو كان تابعاً محلياً؛ فمثلاً يمكنه استدعاء $f(A)$ وانتظار النتيجة R.
2. يستقبل الـ Client Stub طلب الاستدعاء، ويضع اسم التابع f والمعامل A ضمن طلب مناسب للإرسال عبر الشبكة.
3. يُرسل الطلب إلى الـ Server الموجود ضمن Process أخرى.
4. يستقبل الـ Server Stub، الذي يُسمى أيضاً Skeleton، الطلب ويعيد بناءه، فيستخرج اسم التابع والـ Parameters ثم يستدعي التابع الحقيقي $f(A)$ على المخدم.
5. ينفذ التابع العملية المطلوبة وينتج النتيجة R.
6. يجهز الـ Server Stub النتيجة ويرسلها عبر الشبكة إلى الـ Client Stub.
7. يستخرج الـ Client Stub النتيجة ويعيدها إلى الـ Client كما لو كانت ناتجة عن استدعاء محلي.

تتولى لغة IDL أيضاً تحقيق الـ Mapping بين أنماط البيانات المختلفة، فإذا أرسل Client مكتوب بلغة ++C قيمة من النوع $char^*$ إلى Server مكتوب بلغة Java، **يحولها الكود البرمجي المولد إلى String**. وبالمثل، يمكن تحويل int إلى النوع المقابل له، مثل $integer$ أو $long$ ، **وفق اللغة المستخدمة لدى الطرف الآخر**. يسمى الكود البرمجي **الموجود من جهة الزبون Client Stub**، بينما يسمى الكود البرمجي الموجود من **جهة المخدم Server Stub أو Skeleton**. يولد الـ Compiler هذين الجزأين آلياً اعتماداً على وصف الواجهة المكتوب بلغة IDL، وبذلك تصبح عمليات تجهيز الطلب، والإرسال والاستقبال عبر الشبكة، **وإعادة بناء الطلب والجواب شفافة بالنسبة إلى المبرمج**.

فائدة:

- يتضمن توزيع النظام تحديد الخدمات التي تعمل ضمن Processes مستقلة، وطريقة تواصلها، وعدد الموارد المخصصة لها، وكيفية تنفيذها بالتوازي. لذلك يجب أن يتخذ المحلل والمصمم قرار التوزيع، بينما ينفذ المبرمج الخدمة وفق التصميم المحدد. كانت مشكلة RPC أنها جعلت الـ Procedure، التي لا تظهر إلا عند البرمجة، وحدة التوزيع الأساسية.



- **يوفر RPC الشفافية في الاستدعاء،** إذ لا يشعر Client بفرق واضح بين استدعاء تابع محلي موجود ضمن Process نفسها واستدعاء تابع بعيد موجود ضمن Process أخرى. وقد كان RPC أول أسلوب يحقق هذه الشفافية؛ فأصبح الكود المكتوب لاستدعاء تابع يعمل ضمن Mono-Thread بسيطاً ومشابهاً للكود المستخدم لاستدعاء خدمة موزعة على Process أخرى، بينما تتولى البرمجية الوسيطة تنفيذ تفاصيل الاتصال.
- **لا يحتاج Client إلى تحديد طريقة الوصول إلى Server** أو تجهيز الرسالة التي ستُنقل عبر الشبكة، لأن البرمجية الوسيطة تتولى إعداد الطلب وإرساله واستقبال النتيجة.
- **يكون الكود البرمجي الخاص بالـ Client و Server مستقلاً عن نظام التخاطب.** تُوصف واجهة الـ Server باستخدام لغة IDL، ثم يولّد الـ Compiler أثناء الترجمة الكود البرمجي والبنية اللازمين للتواصل، بما في ذلك Client Stub و Server Stub وبروتوكول نقل الطلبات والنتائج.
- **يكون التخاطب في RPC متزامناً Synchronous،** إذ ينتظر الـ Client انتهاء تنفيذ التابع البعيد وإعادة النتيجة.
- **يوجد معيار تستخدمه الشركات المنتجة يسمى OSF/DCE،** وهو اختصار لـ Open Software Foundation/Distributed Computing Environment. يسمح وجود هذا المعيار بمعاملة أكثر من RPC Middleware.

⚡ مساوئ RPC:

نعم	الاستقلال عن منصة العمل
لا	دعم Broadcasting/ point to point
لا	دعم Transactions
نعم	الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط
لا	التصعديّة أو التلاؤم مع تزايد الحركة Traffic
متزامن فقط	دعم التخاطب المتزامن وغير المتزامن.
نعم	دعم بروتوكول تخاطب
لا	دعم الاستدعاء بالاسم
لا	تنوع بنى الاتصال : امكانيّة العمل عن طريق etc, ... wireless, sockets.

لم تحقق هذه التكنولوجيا نجاحاً حقيقياً حيث أن فكرة procedure هي فكرة متدنية المستوى لا تظهر في مرحلة التحليل ولا التصميم وإنما تظهر متأخراً في مرحلة البرمجة مما يضطرنا لتغيير الطرق الحالية في التحليل والتصميم لإظهارها في مراحل متقدمة.

- يمكن تقييم RPC وفق خصائص البرمجيات الوسيطة كما يأتي:
- الاستقلال عن منصة العمل: يدعم الاستقلال عن منصة العمل.
- أنماط الاتصال: يدعم الاتصال Point to Point فقط، ولا يدعم Broadcasting.
- المناقشات Transactions: لا يدعم المناقشات. قد يبدو للمبرمج أنه يستدعي تابعاً عادياً، لكن المناقشة لا تستمر تلقائياً عند انتقال الاستدعاء إلى Process أخرى.
- الوثوقية Reliability: يضمن وصول الرسالة مرة واحدة فقط.
- التصاعدية Scalability: لا يدعم التلاؤم مع تزايد حركة الطلبات.
- نمط التخاطب: يدعم التخاطب المتزامن Synchronous فقط.
- بروتوكول التخاطب: يدعم بروتوكولاً للتخاطب، وهو أول نوع من البرمجيات الوسيطة المدروسة يحقق هذه الخاصية. يستطيع المبرمج استدعاء تابع بعيد بالطريقة نفسها التي يستدعي بها تابعاً محلياً، دون كتابة عمليات Compose و Parse يدوياً.
- الاستدعاء بالاسم: لا يدعم الاستدعاء بالاسم.
- تنوع بنى الاتصال: لا يدعم تنوع بنى الاتصال.

لم تحقق تقنية RPC نجاحاً حقيقياً، لأن وحدة التوزيع فيها هي الـ Procedure، وهي بنية منخفضة المستوى Low-Level لا تظهر في مرحلتي التحليل والتصميم، بل تظهر متأخراً في مرحلة البرمجة. يؤدي ذلك إلى جعل قرار توزيع النظام بيد المبرمج، مع أن التوزيع والمعالجة المتوازية Parallel Processing من القرارات البنيوية التي يجب أن يحددها المحلل والمصمم. ولكي يصبح توزيع الـ Procedures قراراً تصميمياً، يجب إظهارها في مراحل مبكرة، وهذا يفرض تغيير طرائق التحليل والتصميم لتناسب RPC. لذلك ظهرت الحاجة إلى وحدة توزيع أعلى مستوى يمكن تحديدها منذ مرحلة التحليل

فائدة:

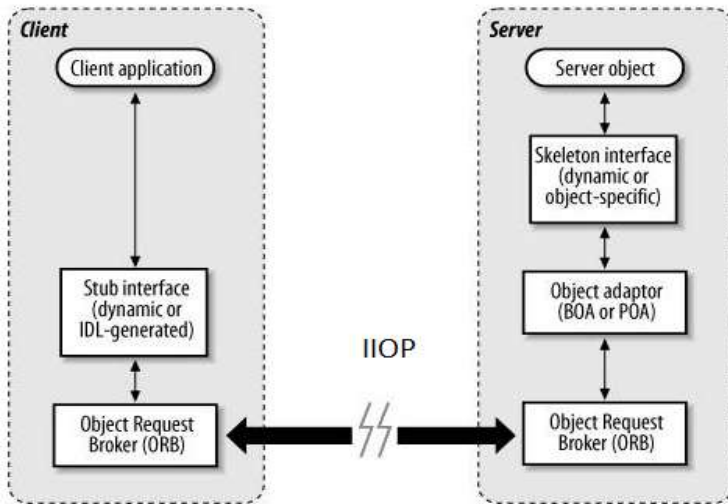
- تم ذكر مثال لنظام كان يعالج نحو مئة ألف Query أو Transaction في الحقيقة، ثم انخفض أداؤه عند الاختبار إلى نحو ست عمليات في الساعة؛ لأن أحد المبرمجين أضاف Synchronized إلى التابع اعتقاداً منه أنها تمنع الأخطاء. أدى ذلك عملياً إلى تحويل التنفيذ المتوازي إلى Mono-Thread، فأصبحت الطلبات تُنفذ بالتتابع. مما يؤكد أن استخدام أدوات التزامن دون الالتزام بتصميم النظام قد يؤدي إلى انهيار الأداء.

«بقدر الكد تُكتسبُ المعالي ... ومن طلب العُلا سهر الليالي»

و تشمل كلاً من:

1. Common Object Request Broker Architecture (CORBA)
2. Component Object Model Plus (COM+)
3. Remote Method Invocation (RMI)
4. Web Services
5. Enterprise JavaBeans (EJB)

1+2. Common Object Request Broker Architecture (CORBA) & Component Object Model Plus (COM+):



- ظهرت البرمجيات الوسيطة غرضية التوجه نتيجة محدودة RPC، التي تستخدم الـ Procedure بوصفها وحدة التوزيع. فال Procedure منخفضة المستوى Low-Level لا تظهر إلا في مرحلة البرمجة، بينما يُفترض أن يكون التوزيع قراراً يتخذه المحلل والمصمم في مراحل مبكرة، ثم ينفذه المبرمج وفق التصميم الموضوع.
- لذلك نحتاج إلى بنية عالية المستوى تظهر في مرحلة التحليل، وتستمر في مرحلة التصميم، ويمكن تحويلها لاحقاً إلى كود برمجي. تتمثل هذه البنية في الغرض Object، الذي يمتلك خصائص البرمجة غرضية التوجه، مثل التغليف Encapsulation والوراثة Inheritance وتحقيق عدة Interfaces.
- في هذا النوع من البرمجيات الوسيطة يصبح الـ Object كاملاً وحدة التوزيع؛ فلا تُوزع كل Procedure داخله بصورة منفصلة، بل يُوزع الغرض بجميع عملياته Methods بوصفه وحدة واحدة. فإما أن يعمل الـ Object كاملاً بصورة محلية Local، وإما أن يعمل كاملاً بصورة بعيدة Remote. وبذلك يستطيع المصمم تحديد الأغراض التي ستعمل محلياً أو عن بُعد، ثم ينفذ المبرمج هذا التصميم، مهما تغير عدد الـ Methods الموجودة داخل الغرض أو أسماؤها.
- يتم التخاطب عن طريق استدعاء Method من Object موجود على المخدم. ويُسمى البرنامج الوسيط الذي يؤمن هذا الاستدعاء Object Middleware أو ORB، وهي اختصار لـ Object Request Broker، أي «وسيط طلبات الأغراض».

تختلف وحدة التوزيع باختلاف نوع البرمجية الوسيطة:

1. في Socket تكون وحدة التوزيع هي التطبيق Application كاملاً.
2. في Queue تكون وحدة التوزيع هي الرسالة Message.
3. في RPC تكون وحدة التوزيع هي الإجراء Procedure.
4. في Object-Oriented Middleware تكون وحدة التوزيع هي الغرض Object.

يوجد معياران أساسيان للبرمجيات الوسيطة غرضية التوجه:

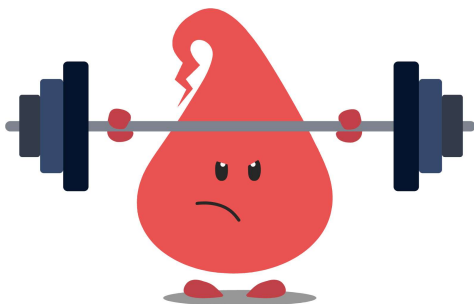
- CORBA: معيار عالمي وضعته OMG، وهي اختصار لـ Object Management Group، وظهر سنة 1990. يرمز CORBA إلى Common Object Request Broker Architecture.
- DCOM: معيار قدمته Microsoft سنة 1996، ويرمز إلى Distributed Component Object Model.

تطورت CORBA لاحقاً تحت تأثير Java؛ إذ أكد المعيار 3.0 مفهوم المكونات البرمجية Components الذي جسده Java من خلال JavaBeans، بهدف تسهيل عملية التطوير والتخفيف من تعقيد CORBA.

✍ محدودية CORBA:

نعم	الاستقلال عن منصة العمل
Point to point	دعم Broadcasting/ point to point
نعم	دعم Transactions
نعم	الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط
لا	التصعيدية أو التلاؤم مع تزايد الحركة Traffic
متزامن فقط	دعم التخاطب المتزامن وغير المتزامن.
نعم	دعم بروتوكول تخاطب
لا	دعم الاستدعاء بالاسم
TCP/ IP	تنوع بنى الاتصال : امكّانة العمل عن طرق etc, ... wireless, sockets
لا تؤمن آليات لإدارة الـ Resources: Fall over, load balancing كما أنها صعبة الاستعمال	

كانت CORBA أول تقنية من البرمجيات الوسيطة غرضية التوجه تسمح بتوزيع الأغراض Objects على آلات أو معالجات مختلفة واستدعاء عملياتها عن بُعد، كما دعمت المناقلات Transactions. إلا أن إعدادها واستعمالها كانا شديدي التعقيد، مما صعب تطبيقها عملياً ودفع إلى البحث عن بديل أسهل.



« لا تحسبنَّ المجدَ تمرّاً أنتَ آكلُهُ
لن تبلغَ المجدَ حتى تلعقَ الصبرا »
عنتر بن شداد

يمكن تقييم CORBA وفق خصائص البرمجيات الوسيطة كما يأتي:

1. الاستقلال عن منصة العمل: تدعم الاستقلال عن منصة العمل.
2. أنماط الاتصال: تدعم الاتصال Point to Point، ولا تدعم Broadcasting.
3. المناقلات Transactions: تدعم المناقلات.
4. الوثوقية Reliability: تضمن وصول الرسالة مرة واحدة فقط.
5. التصاعدية Scalability: لا تدعم التلاؤم مع تزايد حركة الطلبات.
6. نمط التخاطب: تدعم التخاطب المتزامن Synchronous فقط.
7. بروتوكول التخاطب: تدعم بروتوكولاً للتخاطب.
8. الاستدعاء بالاسم: لا تدعم الاستدعاء بالاسم.
9. تنوع بنى الاتصال: تعتمد على TCP/IP ولا تدعم تنوع بنى الاتصال.

تتيح CORBA توزيع الأغراض وتشغيلها على موارد مختلفة، لكنها لا توفر آليات لإدارة هذه الموارد، مثل Load Balancing و Failover. لذلك تقع مسؤولية إعداد التوزيع وإدارته على المطورين، وهو ما يزيد تعقيد استخدامها. وقد تبين من التجارب العملية أن تشغيل CORBA وإعدادها كانا صعبين جداً، وأن تعقيد الاستعمال كان أبرز عيوبها العملية، رغم دعمها توزيع الأغراض والمناقلات.

فائدة:

يذكر أن CORBA قُدمت في التسعينيات بوصفها تقنية ستسمح بتوزيع الأغراض والاستغناء عن التعامل المباشر مع Socket. كانت الفكرة متقدمة، إذ أصبح بالإمكان توزيع Object ودعم المناقلات، لكن تعقيد التقنية جعل استيعابها وتشغيلها صعباً حتى في البيئات البحثية. وعند محاولة استخدامها عملياً، احتاجت إلى جهد كبير لإعدادها وتشغيلها، ولذلك ظهرت الحاجة إلى تقنية أبسط تحقق مفهوم الأغراض الموزعة.

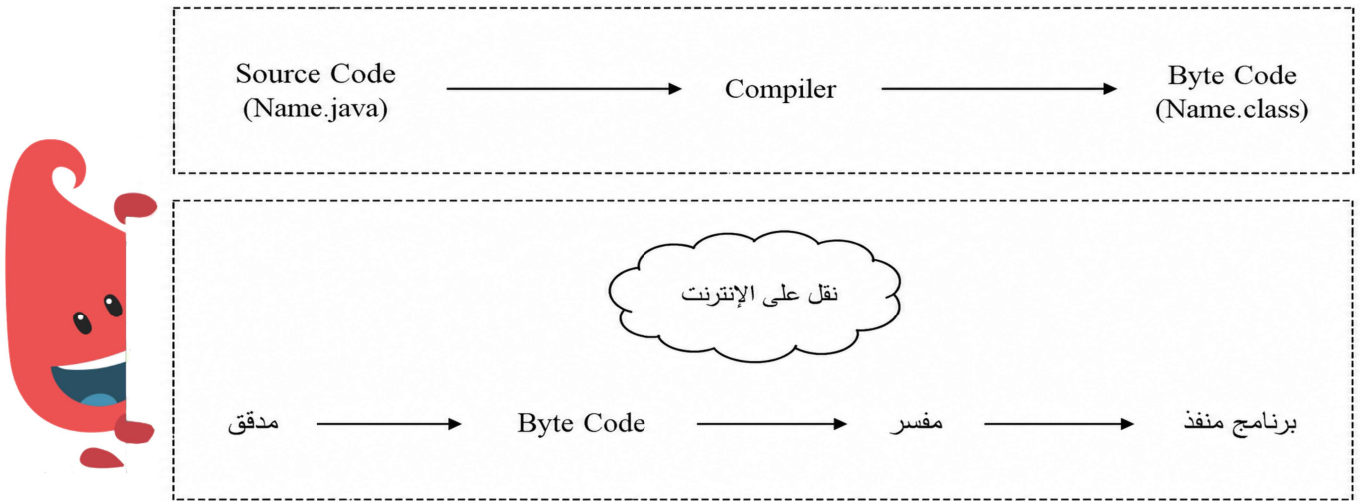
3. RMI & Java

تُعد Java مناسبة لبناء البرمجيات الموزعة؛ لأنها صُممت بحيث يمكن نقل برامجها عبر الشبكة وتشغيلها على آلات مختلفة. وقد ساعدت هذه القابلية على ظهور RMI، وهي اختصار لـ Remote Method Invocation، أي استدعاء عملية Method تابعة لغرض Java موجود على آلة أخرى.

حققت Java قابلية النقل عبر خاصيتين أساسيتين:

- **صغر حجم الكود:**
لا يُحوّل الـ Compiler كود Java مباشرةً إلى تعليمات خاصة بمعالج معين، بل يحوّلها إلى صيغة وسيطة صغيرة الحجم تسمى Bytecode، مما يجعل نقلها عبر الشبكة أسهل ولا يتطلب ضغطها.
- **قابلية التنفيذ على آلات مختلفة:**
لا يرتبط الـ Bytecode بنظام تشغيل أو معالج محدد، ولذلك يمكن تنفيذ الملف نفسه على أي آلة تتوفر عليها بيئة Java والمفسر Interpreter المناسب. ويُعبّر عن ذلك بمبدأ: «اكتب البرنامج مرة واحدة وشغله على آلات مختلفة».

آلية نقل وتنفيذ كود Java:



يُكتب البرنامج أولاً على شكل كود مصدري Source Code ضمن ملف يحمل اللاحقة .java، مثل Name.java. يمثل هذا الملف التعليمات التي كتبها المبرمج بلغة Java، لكنه لا يكون جاهزاً للتنفيذ مباشرةً.

يمر البرنامج بعد ذلك بالمراحل الآتية:

1. الترجمة Compilation:

يقرأ الـ Compiler ملف Name.java ويحوّله إلى Bytecode محفوظ ضمن ملف يحمل اللاحقة .class، مثل Name.class.

2. النقل عبر الشبكة:

يمكن إرسال ملف الـ Bytecode إلى آلة أخرى؛ لأنه لا يحتوي على تعليمات مرتبطة بمعالج أو نظام تشغيل محدد.

3. التحقق Verification:

يفحص المدقق Verifier الـ Bytecode قبل تشغيله للتأكد من سلامة بنيته وعدم احتوائه على تعليمات غير مسموحة.

4. التفسير Interpretation:

يقرأ المفسر Interpreter تعليمات الـ Bytecode ويحوّلها أثناء التشغيل إلى تعليمات تستطيع الآلة المستقبلية تنفيذها.

5. التنفيذ:

يعمل البرنامج على الآلة الجديدة من دون الحاجة إلى إعادة كتابة الكود المصدري أو ترجمته خصباً لنظام تشغيلها.

إذاً، لا تعني قابلية النقل أن الآلات المختلفة تفهم الـ Bytecode مباشرةً، بل تعني أن كل آلة تستخدم بيئة Java المناسبة لها لتفسير الملف الوسيط نفسه وتشغيله.

توزيع الكود والمعطيات

يتكوّن البرنامج من مكونين أساسيين:

1. الكود Code: التعليمات التي تحدد العمليات التي سينفذها البرنامج.
2. المعطيات Data: القيم والمعلومات التي تعالجها هذه التعليمات.

عند توزيع البرنامج على آلتين مختلفتين، يمكن اتباع أحد اتجاهين:

- إما نقل المعطيات إلى الآلة التي يوجد عليها الكود لمعالجتها،
- أو نقل الكود إلى الآلة التي توجد عليها المعطيات لينفذ هناك.

يُعد نقل المعطيات الأسلوب التقليدي الأقدم، وتعود تقنياته إلى ODBC؛ إذ يبقى الكود عند التطبيق، بينما تُرسل الطلبات إلى قاعدة البيانات وتعود المعطيات والنتائج إليه. أما Java فقد أتاحت الاتجاه الآخر أيضاً، وهو نقل الكود البرمجي عبر الشبكة ثم تشغيله على الآلة المستقبلية.

تختلف صورة توزيع البرنامج باختلاف نوع البرمجية الوسيطة:

- في RPC يُقسّم البرنامج إلى إجراءات Procedures يمكن استدعاؤها عن بُعد.
- في Queue يُقسّم إلى تطبيقات Applications مستقلة تتبادل المعطيات على شكل رسائل.
- في Object-Oriented Middleware تكون الأغراض Objects هي العناصر التي يمكن توزيعها واستدعاء عملياتها عن بُعد.

ملاحظة:

عند الحديث عن بنية البرنامج في حالة Queue تكون التطبيقات هي الأجزاء الموزعة، أما عند الحديث عن وحدة التوزيع التي ينقلها Queue بين هذه التطبيقات فتكون الرسالة Message.

في الأنواع السابقة يكون التركيز أساساً على نقل المعطيات بين الأجزاء الموزعة، بينما أتاحت Java نقل الكود نفسه. ويظهر ذلك في نموذج Applet؛ إذ يرسل Client طلباً Request إلى Web Server، فيعيد المخدم صفحة HTML تحتوي على Applet. ينتقل كود الـ Applet مع الصفحة إلى آلة العميل، ثم تتولى بيئة Java الموجودة عليها التحقق منه و تفسيره وتنفيذه.

Java Virtual Machine (JVM)

آلة Java الافتراضية Java Virtual Machine هي بيئة التشغيل التي تتولى قراءة Bytecode وتنفيذه على الآلة المستقبلية. وهي التي تجعل برنامج Java قابلاً للعمل على أنظمة مختلفة؛ إذ تكون لكل منصة JVM مناسبة لها، بينما يبقى ملف الـ Bytecode نفسه مستقلاً عن المنصة.

تكوّن الـ JVM من المكونات الآتية:

1. [المحقق Verifier](#):
يفحص الـ Bytecode قبل تنفيذه للتأكد من صحة بنيته، وسلامة أنماط البيانات، وعدم احتوائه على تعليمات غير مسموحة. وفي حالة الـ Applet تعمل آليات الحماية على منع الكود المحمل من الإنترنت من تنفيذ عمليات خطيرة أو الوصول غير المسموح إلى موارد الجهاز، مثل القرص الصلب.
2. [المفسر Interpreter](#):
يقرأ تعليمات الـ Bytecode ويحوّلها أثناء التشغيل إلى تعليمات تستطيع الآلة الحالية تنفيذها. يسمح ذلك بتشغيل الملف نفسه على منصات مختلفة، لكنه قد يكون أبطأ لأن عملية التفسير تتكرر أثناء التنفيذ.
3. [المكتبات Libraries](#):
مجموعة من الأصناف والعمليات الجاهزة التي يحتاج إليها البرنامج لتنفيذ وظائف شائعة، مثل التعامل مع النصوص والملفات والشبكات وواجهات الاستخدام، بدلاً من كتابة هذه الوظائف من البداية.
4. [الطرائق الأصلية Native Methods](#):
طرائق لا تكون مكتوبة بالكامل بلغة Java، بل بلغة مرتبطة بالمنصة مثل C أو ++C، وتُستخدم عندما يحتاج البرنامج إلى التعامل المباشر مع نظام التشغيل أو العتاد أو إحدى المكتبات الأصلية.
5. [جامع النفايات Garbage Collector](#):
يدير الذاكرة آلياً؛ إذ يتعرّف إلى الأغراض Objects التي لم يعد البرنامج قادراً على الوصول إليها، ثم يحرر الذاكرة التي كانت تشغلها. وبذلك لا يحتاج المبرمج إلى تحرير ذاكرة كل غرض يدوياً.
6. [المتّرجم الأنبي JIT](#):
وهو اختصار لـ Just-In-Time Compiler . يترجم الأجزاء كثيرة التنفيذ من الـ Bytecode إلى تعليمات آلة أصلية Native Code أثناء تشغيل البرنامج، بحيث تُنفذ مباشرةً بدل إعادة تفسيرها في كل مرة، مما يحسن سرعة الأداء.

تحميل المعطيات باستخدام Applet

لا تقتصر الـ Applet على تنفيذ الكود الذي انتقل إليها مع صفحة HTML، بل يمكنها أيضاً استخدام عنوان URL للوصول إلى مورد موجود على الإنترنت وتحميل محتواه. فالتعليمة:

```
new URL("http://www.sun.com/obj").getContent();
```

تنشئ عنواناً يشير إلى المورد obj، ثم تستدعي `getContent()` لجلب محتواه عبر الشبكة. وبذلك تستطيع الـ Applet تحميل معطيات أو قيم إضافية تحتاج إليها أثناء التنفيذ، مع بقائها خاضعة لقيود الحماية التي تفرضها بيئة Java.

Remote Method Invocation (RMI)

- دفعت صعوبة استخدام CORBA إلى البحث عن طريقة أسهل لتوزيع الأغراض، فظهرت RMI في Java. يرمز RMI إلى [Remote Method Invocation](#)، أي «استدعاء طريقة عن بُعد»، وهي آلية تسمح لأغراض Java الموجودة ضمن آلتها JVM مختلفتين بالتخاطب عن طريق استدعاء الـ Methods.
- لجعل غرض Java قابلاً للاستدعاء عن بُعد، يحقق الغرض واجهة تسمى Remote Interface. تحدد هذه الواجهة العمليات التي يُسمح للعميل باستدعائها عبر الشبكة، من دون أن يحتاج إلى معرفة مكان تنفيذها أو تفاصيل الاتصال بها. لذلك يستطيع الـ Client استدعاء Method تابعة للغرض البعيد بطريقة مشابهة لاستدعاء Method من غرض محلي.
- تختلف RMI عن CORBA، وفق النموذج المدروس، في أنها تسمح بتمرير الأغراض Objects بوصفها Parameters بين طرفي الاتصال، بينما لا تنتقل أغراض CORBA نفسها، بل تتبادل الأطراف أنماطاً أبسط من المعطيات. ولكي يمكن نقل غرض Java عبر الشبكة، يجب تحويل حالته إلى صيغة متتابعة من البايتات، وتسمى هذه العملية Serialization. يعيد الطرف المستقبل بناء الغرض من هذه البيانات بعملية عكسية تسمى Deserialization.
- عملياً، إذا كان الغرض غرضاً بعيداً Remote Object، يُمرر عادةً مرجع باستدعائه عن بُعد. أما الأغراض العادية القابلة للـ Serialization، فيمكن تمرير نسخة من حالتها إلى الطرف الآخر.

بنيان Java RMI:

يتكوّن بنيان Java RMI من العناصر الآتية:

1. Client Application أو Applet:

التطبيق الذي يطلب استخدام خدمة موجودة على آلة أخرى.

2. Server Object:

الغرض الحقيقي الموجود على المخدم، والذي يحقق الـ Remote Interface وينفذ العمليات المطلوبة.

3. RMI Registry:

سجل يحتفظ برابط اسم منطقي للغرض البعيد بالمرجع الذي يسمح بالوصول إليه. يسجل الـ Server غرضه في السجل، ثم يبحث الـ Client عنه باسمه. يقتصر دور الـ Registry على تسجيل الغرض والعثور عليه، أما استدعاء طرائقه بعد ذلك فيتم عبر طبقات RMI الآتية:

1. Stub/Skeleton Layer:

تمثل الواجهة الوسيطة بين العميل و المخدم. يكون الـ Stub وكيلاً محلياً Proxy عن الغرض البعيد ويحقق الواجهة نفسها؛ لذلك يتعامل معه الـ Client كما لو كان الغرض موجوداً لديه. أما الـ Skeleton فيستقبل الطلب من جهة المخدم ويوجهه إلى الغرض الحقيقي.

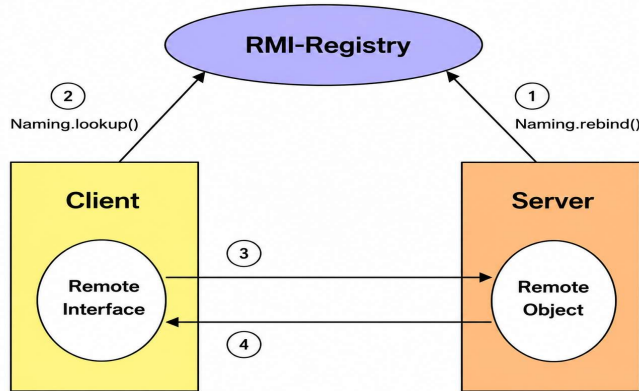
2. Remote Reference Layer

تدير المراجع الخاصة بالأغراض البعيدة، وتحدد الغرض المقصود بكل استدعاء.

3. Transport Layer

تتولى فتح الاتصال عبر الشبكة ونقل الطلبات والنتائج بين آلتَي الـ JVM.

تسلسل الوصول إلى الغرض البعيد



1. تسجيل الغرض:

ينشئ الـ Server غرضًا يحقق الـ Remote Interface، ثم يسجله باسمه في الـ RMI Registry باستخدام Naming.rebind().

2. البحث عن الغرض:

يبحث الـ Client عن الغرض باسمه داخل الـ Registry باستخدام Naming.lookup()، فيحصل على Stub يمثل الغرض البعيد ويحقق الواجهة نفسها.

3. إرسال الاستدعاء:

يستدعي الـ Client إحدى الـ Methods من خلال الـ Stub، فينتقل الطلب عبر طبقات RMI إلى الـ Remote Object الحقيقي الموجود على الـ Server.

4. إعادة النتيجة:

ينفذ الـ Remote Object العملية المطلوبة، ثم تعود النتيجة عبر الطبقات نفسها إلى الـ Client.

لا يحزنك أنك فشلت ما دمت تحاول الوقوف على قدميك من جديد

مراحل بناء التخابط باستعمال RMI

- يبدأ بناء التخابط باستخدام RMI بإنشاء واجهة للغرض المخدم، وهي Java Interface تحتوي على ترويسات الطرائق التي يمكن استدعاؤها عن بُعد، وتُشتق من الواجهة Remote. بعد ذلك يُبنى صف الغرض المخدم الذي يحقق هذه الواجهة؛ فمثلاً يمكن إنشاء صف باسم MyObject يحقق Remote ويحتوي على الطرائق f() و g() و k().
- يُستخدم javac لترجمة البرنامج والحصول على Bytecode، ثم تُستخدم الأداة rmic لتوليد الـ Stub والـ Skeleton اللازمين للتخابط. بعد ذلك ينشئ المخدم نسخة Instance من صف الغرض المخدم، مثل:

```
MyObject x = new MyObject();
```

- ثم يسجل هذه النسخة باسم محدد لدى الـ Registry باستخدام Naming.rebind()، فمثلاً يمكن ربط الغرض x بالاسم hello:

```
Naming.rebind("hello", x);
```

- كما يُنشأ ويُوطن Security Manager لحماية المخدم، ثم يشغل مخدم الأسماء Registry على آلة المخدم، ويشغل برنامج المخدم الذي يسجل الغرض، وبعده يشغل تطبيق العميل.
- يبحث العميل في الـ Registry عن الغرض المسجل باستخدام اسمه:

```
Remote a = (Remote) Naming.lookup("hello");
```

- فيحصل على مرجع بعيد للغرض، ثم يستطيع استدعاء طرائقه، مثل: `a.f()` , `a.g()`;

تعد الـ Registry عملية مستقلة وثالثة إلى جانب عمليتي العميل والمخدم، وتؤدي دور الوسيط الذي يربط اسم الغرض بمرجه، ليتمكن العميل من الوصول إليه واستدعائه عن بعد. وقد ساعدت بساطة Java وكثرة مكتباتها وواجهاتها على جعل RMI أسهل في الاستخدام من CORBA و ++C.

⚡ خصائص RMI ومحدوديتها:

نعم	الاستقلال عن منصة العمل
Point to point	دعم Broadcasting/ point to point
لا	دعم Transactions
نعم	الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط
لا	التصعديّة أو التلاؤم مع تزايد الحركة Traffic

متزامن فقط	دعم التخاطب المتزامن وغير المتزامن.
نعم	دعم بروتوكول تخاطب
نعم	دعم الاستدعاء بالاسم
TCP/IP	تنوع بنى الاتصال : امكأة العمل عن طرَق etc, ... wireless, sockets.
مع أن RMI لا تدعم المناقلات إلا أنها انتشرت بسبب سهولة استعمالها	

يمكن تقييم RMI وفق خصائص البرمجيات الوسيطة كما يأتي:

- الاستقلال عن منصة العمل: تدعم الاستقلال عن منصة العمل؛ لأنها تعمل ضمن بيئة Java.
- أنماط الاتصال: تدعم الاتصال Point to Point فقط، ولا تدعم Broadcasting.
- المناقلات Transactions: لا تدعم المناقلات، وتُعد هذه الخاصية أبرز محدودياتها.
- الوثوقية Reliability: تضمن وصول الرسالة مرة واحدة فقط.
- التصاعدية Scalability: لا تدعم التلاؤم مع تزايد حركة الطلبات.
- نمط التخاطب: تدعم التخاطب المتزامن Synchronous فقط.
- بروتوكول التخاطب: تدعم بروتوكولاً للتخاطب.
- الاستدعاء بالاسم: تدعم الاستدعاء بالاسم من خلال تسجيل الغرض والبحث عنه ضمن الـ Registry.
- تنوع بنى الاتصال: تعتمد على TCP/IP، ولا تدعم تنوع بنى الاتصال.

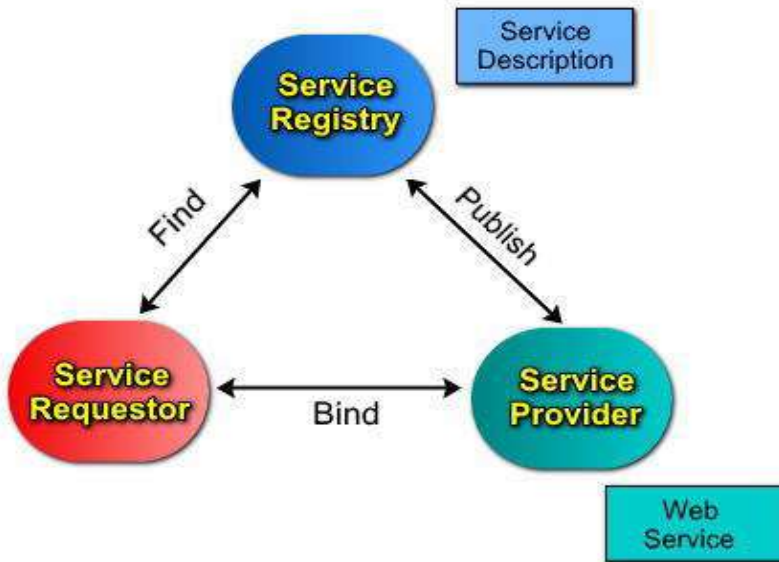
تتميز RMI بسهولة استخدامها؛ لأنها تسمح للمبرمج باستدعاء طرائق الغرض البعيد بطريقة قريبة من استدعاء طرائق الغرض المحلي. ولهذا انتشرت على الرغم من عدم دعمها للمناقلات Transactions، التي تُعد أبرز عيوبها.

لذلك ظهرت الحاجة إلى تقنية تجمع بين دعم المناقلات الموجود في CORBA وسهولة الاستخدام الموجودة في RMI، وهو ما مهّد للانتقال إلى Enterprise JavaBeans (EJB).

4. Web Service

تُعد الـ Web Service أحد تطبيقات البرمجيات الوسيطة التي تسمح لتطبيقات موجودة ضمن Processes مختلفة بالتخاطب عبر الشبكة. حيث تعتمد تعتمد على نقل الرسائل المكتوبة بصيغة XML باستخدام بروتوكول HTTP، ويُعبّر عن ذلك بـ XML over HTTP. تحدد XML بنية البيانات المرسل، بينما يتولى HTTP نقلها بين طالب الخدمة ومقدمها.

يتم تشبيه ال Web Service بال Socket؛ فكلهما يتيح التواصل بين تطبيقات مستقلة، لكن ال Web Service توفر طريقة أكثر تنظيماً وملاءمة للتطبيقات القابلة للتوسع. كما يمكن استخدامها بوصفها إحدى الوسائل التي ينتقل عبرها طلب استدعاء خدمة بعيدة، إلى جانب SOAP و HTTP.



يتكوّن المخطط من ثلاثة أطراف:

- **Service Provider**: الجهة التي تنشئ ال Web Service وتقدم عملياتها.
- **Service Registry**: سجل يحتوي على وصف الخدمات المنشورة ومعلومات الوصول إليها.
- **Service Requestor**: التطبيق الذي يبحث عن خدمة ويريد استخدامها.

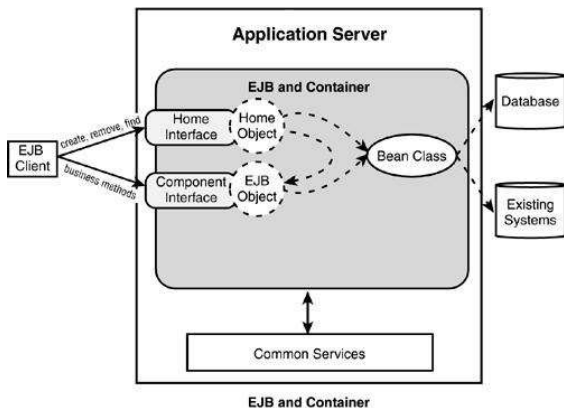
تتم آلية العمل كما يأتي:

1. **Publish**: ينشر ال Service Provider وصف خدمته Service Description في ال Service Registry.
2. **Find**: يبحث ال Service Requestor في السجل عن الخدمة المناسبة ويحصل على وصفها ومعلومات مقدمها.
3. **Bind**: يستخدم طالب الخدمة المعلومات التي حصل عليها للتواصل مع ال Service Provider واستدعاء ال Web Service.

■ **لا ينفذ** ال Service Registry الخدمة نفسها، بل يساعد طالب الخدمة على العثور على مقدمها، ثم يحدث التخابط الفعلي بين ال Service Requestor وال Service Provider.

■ تدعم ال Web Service التصاعدية Scalability، لكنها لا تدعم الاستدعاء بالاسم ولا المناقلات Transactions؛ لذلك فإن تنفيذ Rollback محلياً لا يؤدي تلقائياً إلى التراجع عن العمليات المنفذة في نظام آخر عبر Web Service، ما لم توجد آلية مستقلة لإدارة مناقلة موزعة.

5. EJB & J2EE



ظهرت Enterprise JavaBeans (EJB) استجابةً للحاجة إلى تقنية تجمع بين سهولة RMI ودعم المناقلات Transactions الموجود في CORBA. تعتمد EJB على RMI over IIOP؛ إذ إن IIOP هو البروتوكول الذي طورته CORBA ويدعم المناقلات فأخذت EJB سهولة الاستدعاء البعيد من RMI ودعم المناقلات من CORBA. لكنها لم تقتصر على الجمع بينهما، بل أضافت أيضاً إدارة دورة حياة الأغراض والموارد.

بنية EJB وإدارة الموارد:

- في RMI ينشئ المخدم Instance من الغرض ثم يسجلها باسم محدد. وعند الحاجة إلى عدة Instances تعمل بالتوازي، يجب إنشاء كل واحدة منها وتسجيلها والتنسيق للوصول إليها. أما في EJB، فلا يُسجل Object محدد، بل يُسجل Class الذي يمكن إنشاء الأغراض منه.
- يبحث Client عن Class من خلال Home Interface، ثم يطلب إنشاء Instance جديدة. لا تعني عملية new هنا أن الغرض يُنشأ على آلة العميل، بل يكون الإنشاء الفعلي داخل Application Server. ويتولى EJB Container إنشاء Instance وإدارتها، بينما يحصل العميل على واجهة تتيح له استخدام الغرض عن بُعد.

تظهر في المخطط واجهتان أساسيتان:

Home Interface

تُستخدم لتنفيذ العمليات المرتبطة بالغرض مثل إنشائه أو حذفه أو البحث عنه.

Component Interface

تُستخدم لاستدعاء Business Methods التي يقدمها الغرض.

- داخل Container يوجد Home Object و EJB Object و Bean Class، ويتولى Container الربط بينها وإدارة الأغراض. كما يوفر Common Services، ويربط المكونات بقواعد البيانات Database والأنظمة الموجودة مسبقاً Existing Systems.
- عندما ينتهي العميل من استخدام Instance، لا يلزم إتلافها مباشرة، بل يمكن للContainer الاحتفاظ بها وإعادة استخدامها لخدمة طلب آخر. تسمى هذه الآلية Pooling. فإذا طلب عشرة عملاء أغراضاً بصورة متوازية، يمكن إنشاء عشر Instances، وبعد انتهاء الطلبات تصبح هذه Instances متاحة لإعادة الاستخدام بدل إنشاء أغراض جديدة في كل مرة.
- يشبه ذلك عدد الموظفين الموجودين في صالة؛ إذ تُوزع الطلبات على الموظفين المتاحين، وعندما ينتهي أحدهم من طلب يُعطى طلباً جديداً. ويمكن تحديد حد أدنى Min وحد أعلى Max لعدد Instances؛ فيزيدها الوسيط عند ارتفاع الضغط ويخفضها عند انخفاضه. وبذلك تصبح EJB برمجة بسيطة ووسيلة لإدارة الموارد في الوقت نفسه.
- تكون وحدة التوزيع في RMI و CORBA هي Object، بينما تصبح وحدة التوزيع في EJB هي Class. لذلك لا يحتاج العميل والمخدم إلى التنسيق مسبقاً حول عدد Instances المطلوبة؛ فالContainer هو الذي ينشئها ويديرها ويعيد استخدامها وفق الضغط والإعدادات المحددة.

⚡ خصائص EJB و J2EE:

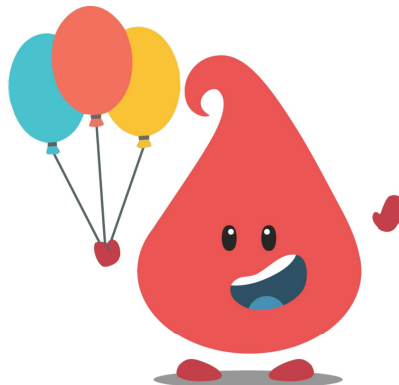
نعم	الاستقلال عن منصة العمل
نعم باستعمال MDB و Session Bean	دعم Broadcasting/ point to point
نعم فهو يستعمل RMI over IIOP	دعم Transactions
نعم	الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط

نعم	التصعيدية أو التلاؤم مع تزايد الحركة Traffic
نعم	دعم التخاطب المتزامن وغير المتزامن.
نعم	دعم بروتوكول تخاطب
نعم	دعم الاستدعاء بالاسم
نعم	تنوع بنى الاتصال : امكأة العمل عن طرّق etc, ... wireless, sockets
IIOP هو البروتوكول الذي طورته CORBA و الذي يدعم المناقلات، حيث يجمع EJB بين سهولة الـ RMI و دعم CORBA للمناقلات	

يمكن تقييم EJB و J2EE وفق خصائص البرمجيات الوسيطة كما يأتي:

1. الاستقلال عن منصة العمل: تدعمان الاستقلال عن منصة العمل.
2. أنماط الاتصال: تدعمان Point to Point و Broadcasting باستخدام Session Bean و MDB.
3. المناقلات Transactions: تدعمان المناقلات باستخدام RMI over IIOP.
4. الوثوقية Reliability: تضمنان وصول الرسالة مرة واحدة فقط.
5. التصاعدية Scalability: تدعمان التلاؤم مع تزايد حركة الطلبات، ويساعد الـ Pooling وإدارة عدد الـ Instances على تحقيق ذلك.
6. نمط التخاطب: تدعمان التخاطب المتزامن وغير المتزامن.
7. بروتوكول التخاطب: تدعمان بروتوكولاً للتخاطب.
8. الاستدعاء بالاسم: تدعمان الاستدعاء بالاسم.
9. تنوع بنى الاتصال: تدعمان بنى اتصال متعددة.

وهكذا جمعت EJB بين سهولة RMI ودعم CORBA للمناقلات، وأضافت إليهما إدارة الموارد والـ Instances داخل الـ Container.



الأدوار في EJB:

تدعم EJB هندسة البرمجيات من خلال توزيع العمل بين ستة أدوار مستقلة:

1. Bean Provider
2. Bean Assembler
3. Deployer
4. Container Provider
5. Application Server Provider
6. Administrator

■ ملخص لما سبق :

EJB/J2EE	RMI	CORBA	RPC	Queue	3 Tiers Client/Server	Client/ Server	خصائص البرمجيات الوسيلة
نعم	نعم	نعم	نعم	يتوفر على منصات: Unix, IBM/OS2 , Windows NT	نعم	نعم	الاستقلال عن منصة العمل
نعم يستعمل Session Bean و	Point to point only	Point to point only	لا	نعم	Point to point only	Point to point only	دعم Broadcasting/ point to point
نعم يستعمل RMI over IIOP	لا	نعم	لا	نعم	لا	لا	دعم Transactions
نعم	نعم	نعم	نعم	يضمن وصول الرسائل و ذلك بالمحافظة على نسخة على القرص الصلب لا تهدم ما لم تعالج الرسالة	يعتمد على المبرمج	يعتمد على المبرمج	الوثوقية: أن تصل الرسالة مرة ومرة واحدة فقط
نعم	لا	لا	لا	نعم	نعم	لا	التصعيدية أو التلاؤم مع تزايد الحركة Traffic
نعم	متزامن فقط	متزامن فقط	متزامن فقط	نعم	متزامن فقط	متزامن فقط	دعم التخاطب المتزامن وغير المتزامن.
نعم	نعم	نعم	نعم	لا	لا	لا	دعم بروتوكول تخاطب
نعم	نعم	لا	لا	لا	لا	لا	دعم الاستدعاء بالاسم
نعم	TCP/IP	TCP/IP	لا	نعم	socket	socket	تنوع بني الاتصال : امكانة العمل عن طرُق wireless,sockets ... ,etc

وحدة التوزيع	البرمجة الوسيطة
التطبيق Application	Socket
الرسالة Message	Queue
الإجراء Procedure	RPC
الكائن Object	CORBA & RMI
الصف Class	EJB